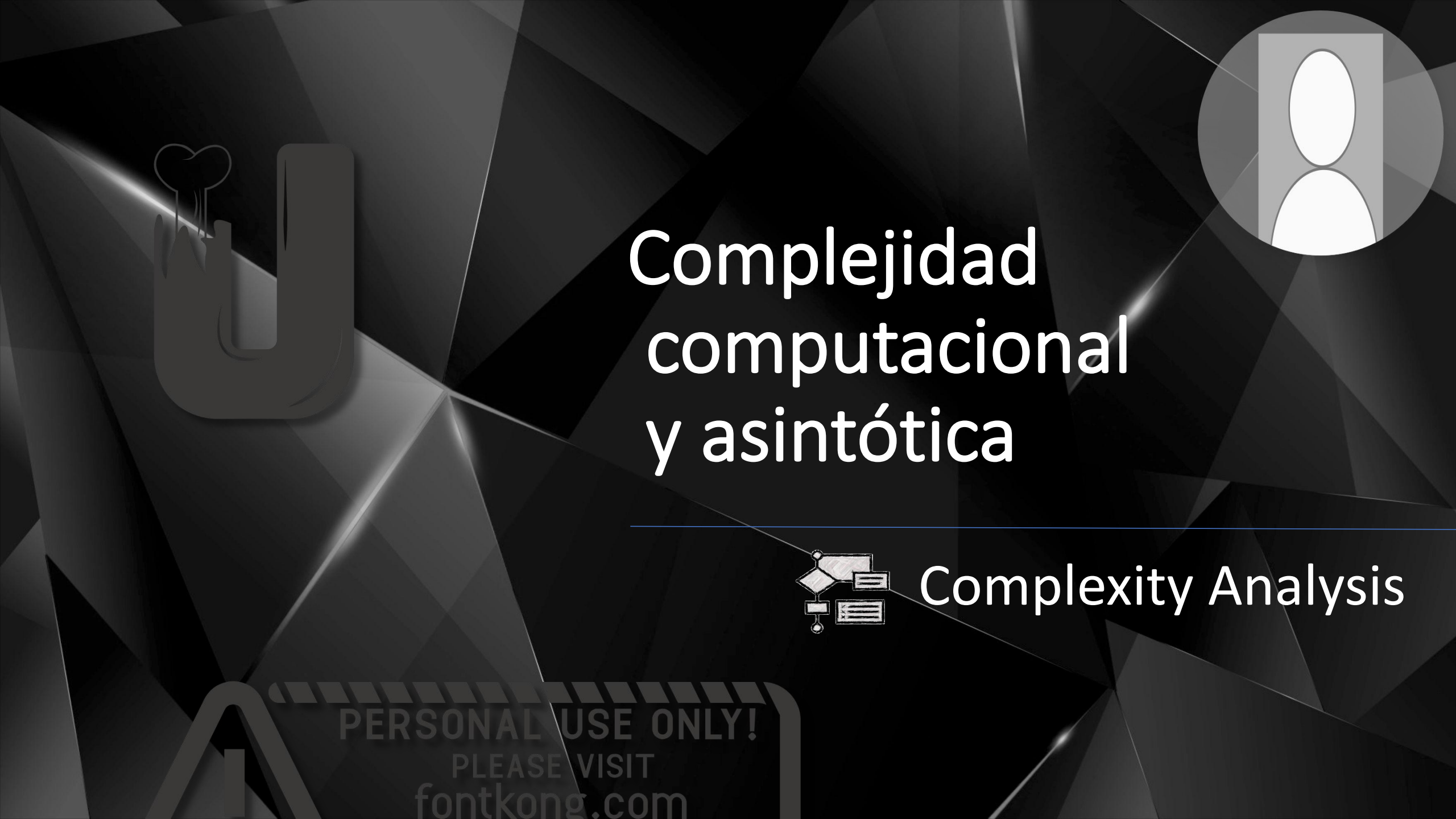
U1

Algoritmos y Estructuras de Datos III

Análisis & Complejidad Computacional



Complejidad computacional y asintótica

Complexity Analysis

Complejidad computacional y asintótica



CORE

ANALISIS DE LA COMPLEJIDAD
ANALISIS ASINTOTICO

ANALISIS DE RENDIMIENTO

ANALISIS

ANALISIS DE ALGORITMOS
ANALISIS ASINTOTICO
PONDERACION ,PEOR PROMEDIO, MEDIO Y MEJOR CASO
MOTACIONES ASINTOTICAS

BIG-O

COMPLEJIDAD

COMPLEJIDAD EN EL TIEMPO
COMPLEJIDAD EN EL ESPACIO

COMPARATIVA TIEMPO & ESPACIO

ENTENDIENDO BIG-O PICTURE

Análisis de algoritmos | Análisis de Complejidad



Análisis de la complejidad



Por qué un Análisis de Complejidad?

Un mismo problema se puede resolver con frecuencia con algoritmos que difieren en eficiencia. Las diferencias entre los algoritmos pueden ser irrelevantes para procesar un pequeño número de elementos de datos, pero estas diferencias crecen con la cantidad de datos. Para comparar la eficiencia de los algoritmos, **Juris Hartmanis y Richard E. Stearns** desarrollaron una medida del grado de dificultad de un algoritmo llamado **complejidad computacional**.
La complejidad computacional indica cuánto **esfuerzo** se necesita para aplicar un algoritmo o qué tan **costoso** es.
Este costo se puede medir de varias maneras, y el contexto particular determina su significado. En esta Unidad abordaremos los dos criterios de eficiencia: **el tiempo y el espacio**. El factor tiempo suele ser más importante que el del espacio, por lo que las consideraciones de eficiencia suelen centrarse en la cantidad de tiempo transcurrido al procesar los datos. Sin embargo, el algoritmo más ineficiente que se ejecuta en una SuperComputadora puede ejecutarse mucho más rápido que el algoritmo más eficiente que se ejecuta en una PC, por lo que el tiempo de ejecución siempre depende del sistema.
Por ejemplo, para comparar 100 algoritmos, todos ellos tendrían que ejecutarse en la misma máquina. Además, los resultados de las pruebas en tiempo de ejecución dependen del idioma en el que se escribe un algoritmo determinado, incluso si las pruebas se realizan en la misma máquina. Si los programas se compilan, se ejecutan mucho más rápido que cuando se interpretan. Un programa escrito en C o C++ puede ser más rápido que el mismo programa codificado en Java o C#.

Algoritmo A



Algoritmo B



Algoritmo C



¿Cuál es el mejor algoritmo?

Algoritmo A



Memoria

Velocidad

Algoritmo B



Memoria

Velocidad

Algoritmo C



Memoria

Velocidad

Como Medimos la Complejidad?





Complejidad en el tiempo

Introducción



Veamos una alegoría para entenderlo...

Tres opciones (***algoritmos***) para llevar
para enviar los archivos de texto



Internet



Ave



Cohete

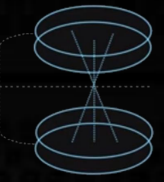
Algoritmo A



Algoritmo B



Algoritmo C



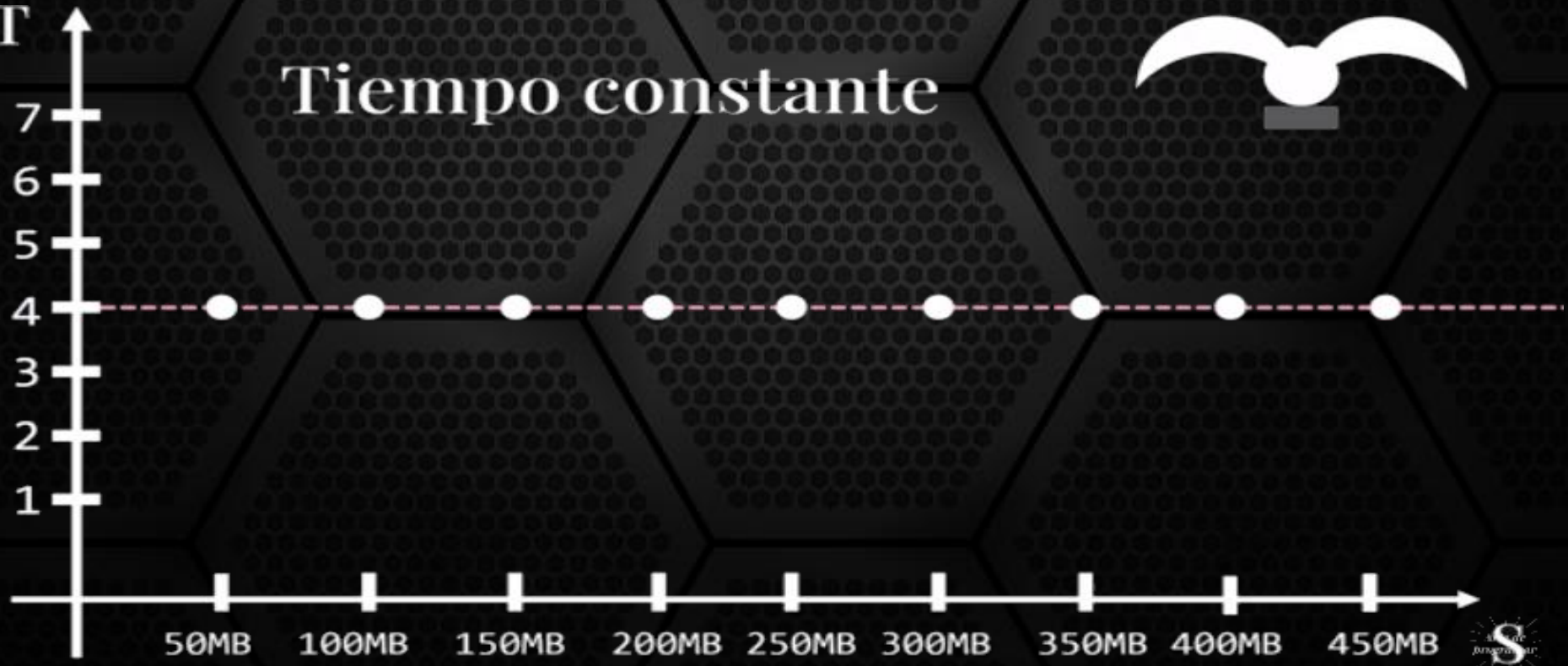
Ave



Si el tamaño de los archivos aumenta

El ave siempre tardará lo mismo en llegar

Tiempo constante



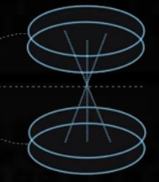
Algoritmo A



Algoritmo B



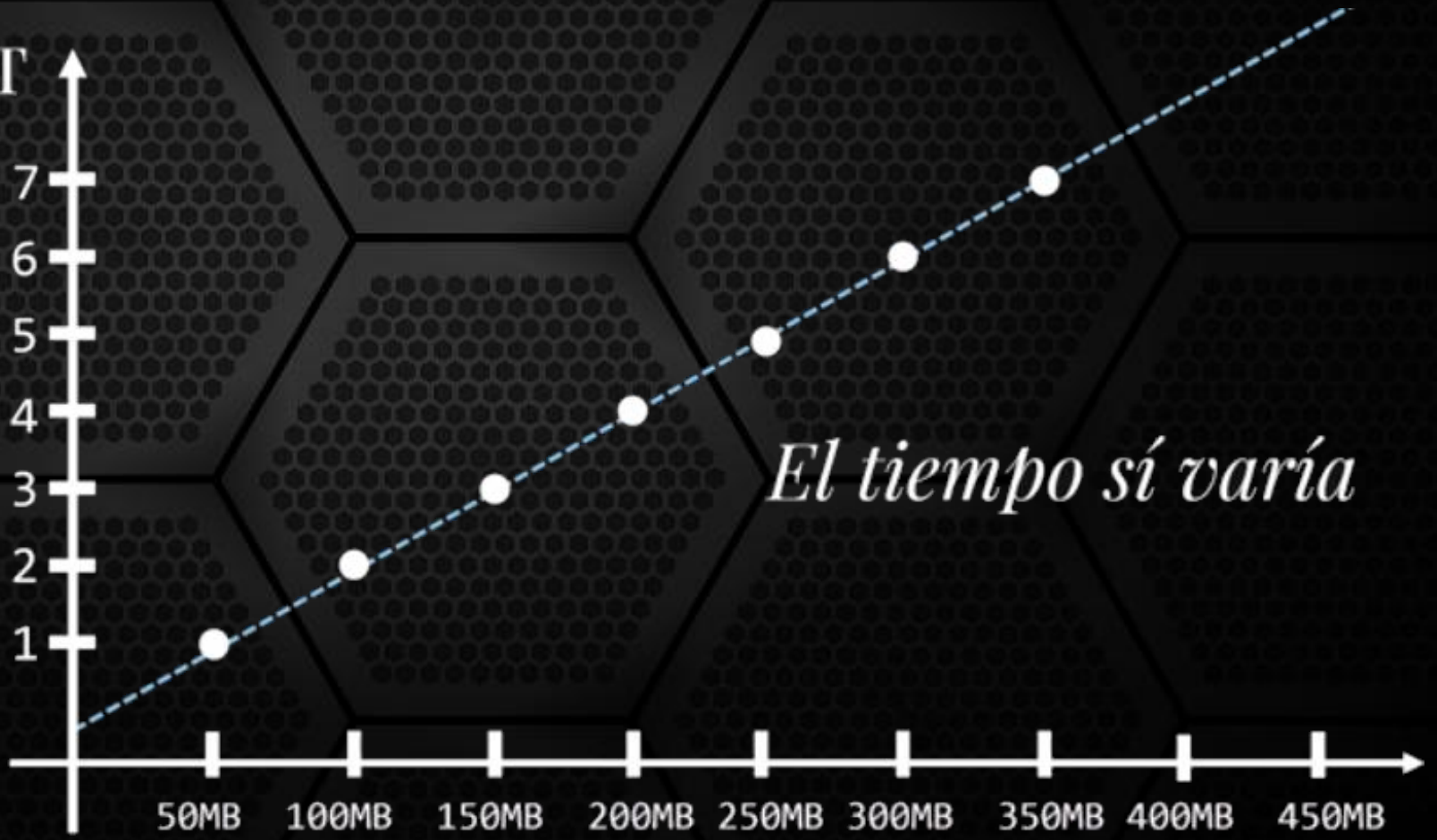
Algoritmo C



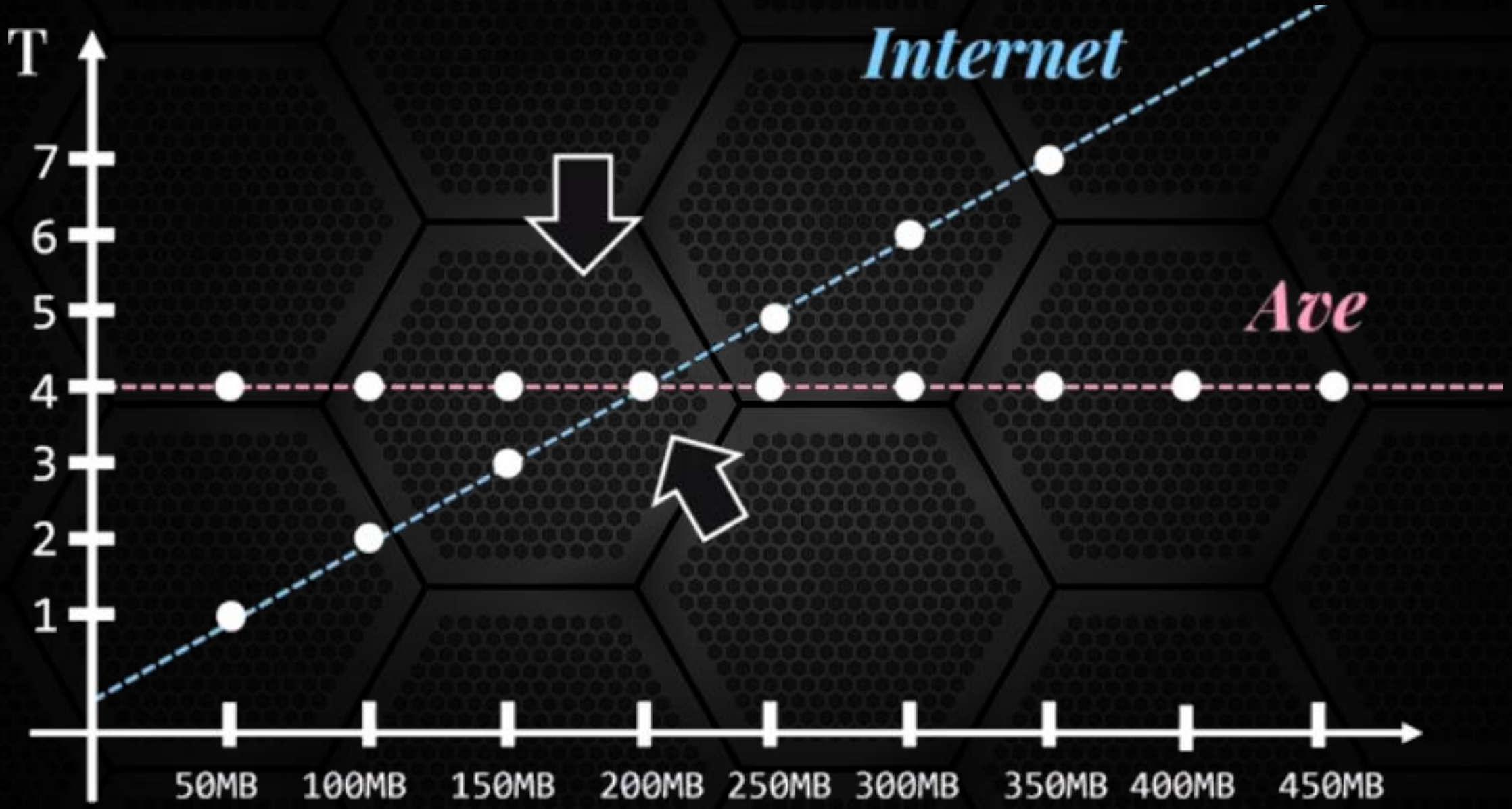
Internet

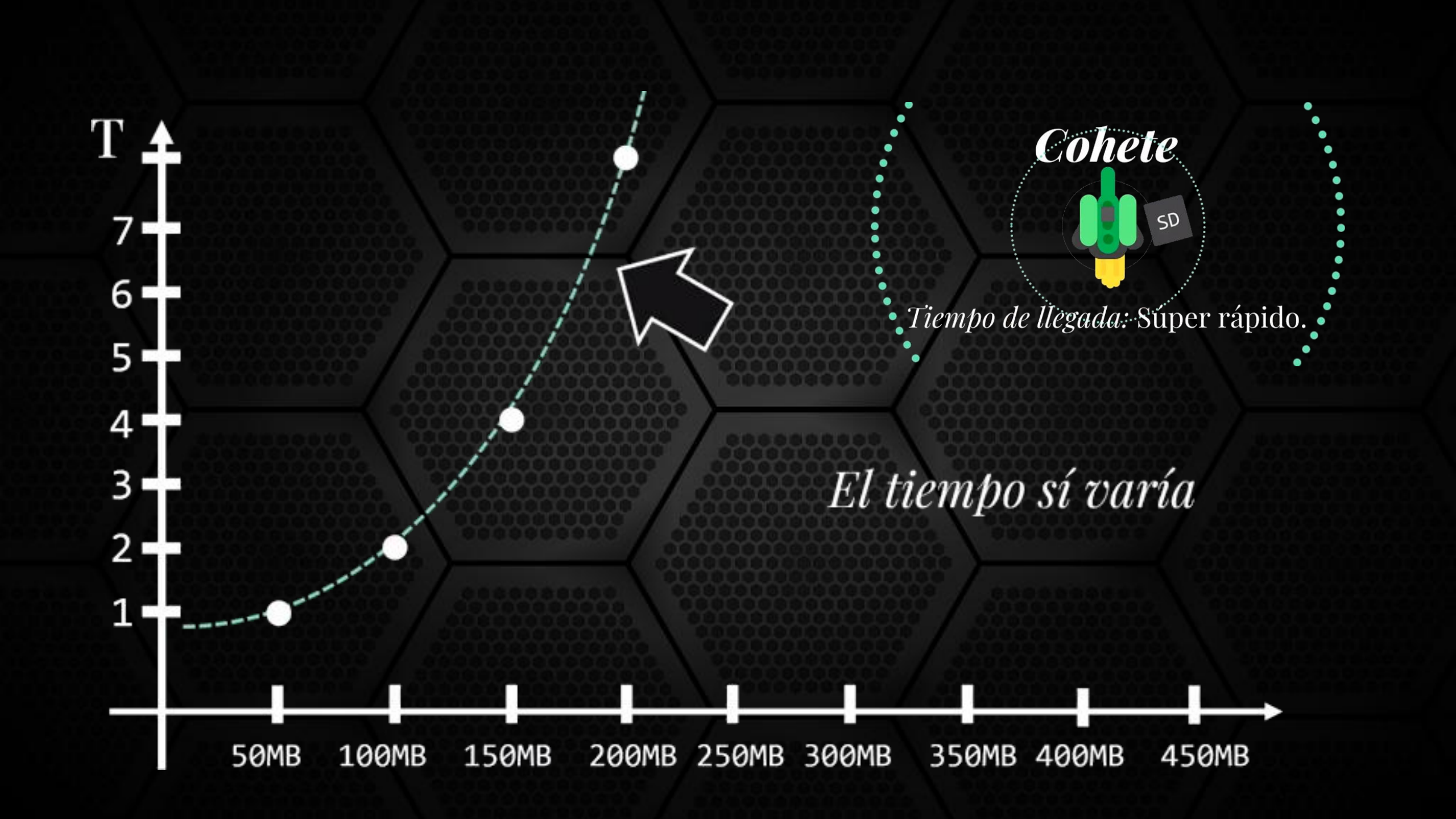


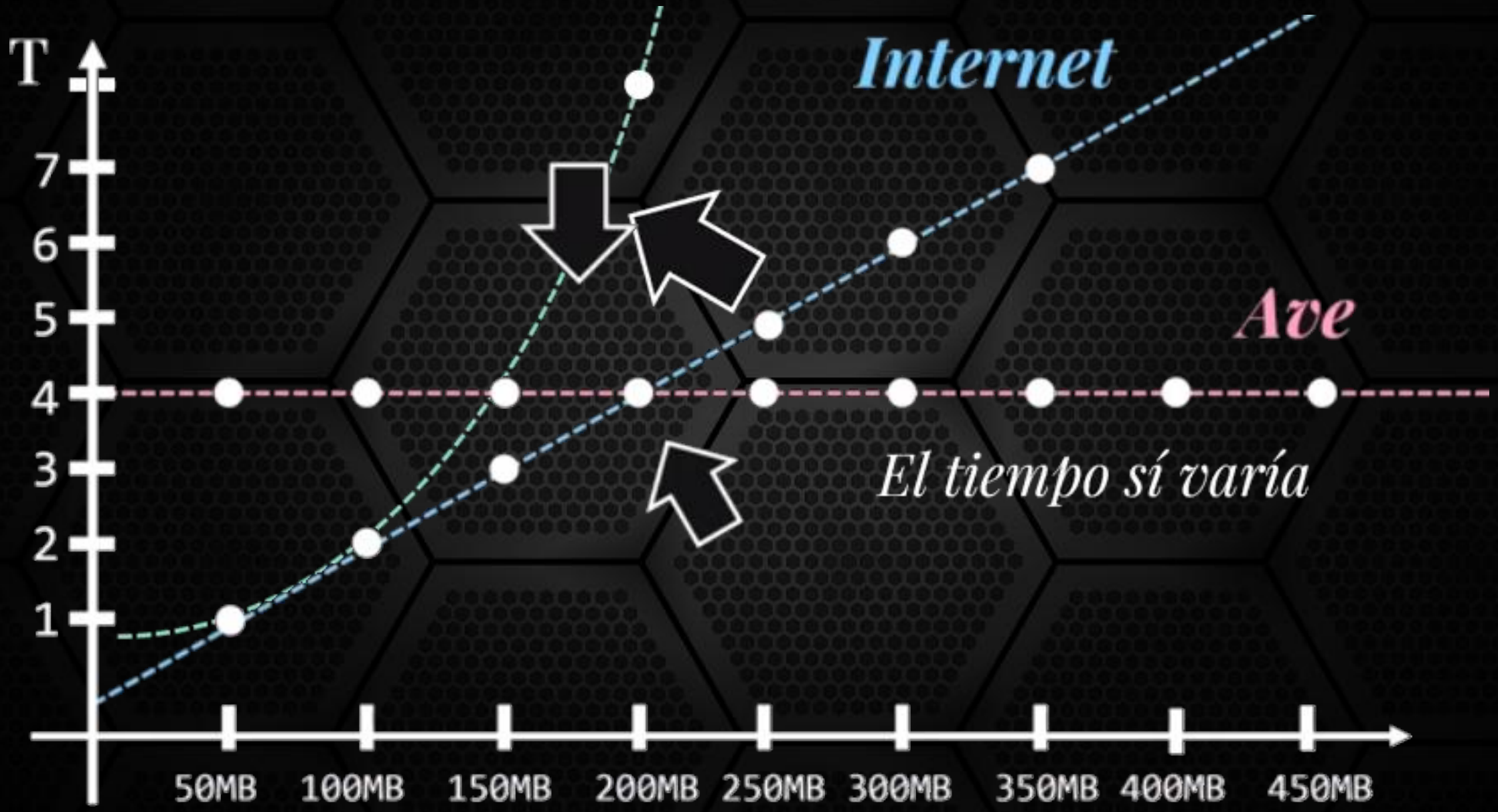
Tiempo de llegada: Depende de la velocidad (`0.1 Mbps`)



El tiempo sí varía


T
7
6
5
4
3
2
1
Internet
Ave
50MB
100MB
150MB
200MB
250MB
300MB
350MB
400MB
450MB

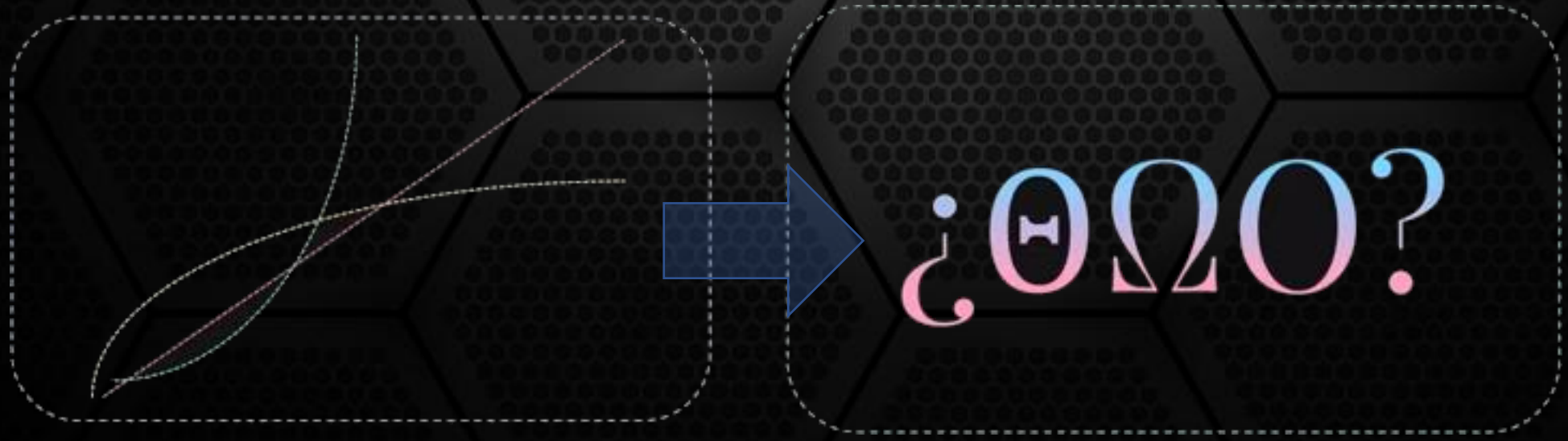

T
7
6
5
4
3
2
1
50MB
100MB
150MB
200MB
250MB
300MB
350MB
400MB
450MB
Cohete
SD
Tiempo de llegada: Súper rápido.
El tiempo sí varía


T
7
6
5
4
3
2
1
Internet
Ave
El tiempo sí varía
50MB
100MB
150MB
200MB
250MB
300MB
350MB
400MB
450MB

Complejidad en el tiempo

Es la cantidad de tiempo que un algoritmo requiere cuando los datos de entrada se incrementan





Análisis de la complejidad (complejidad asintótica)

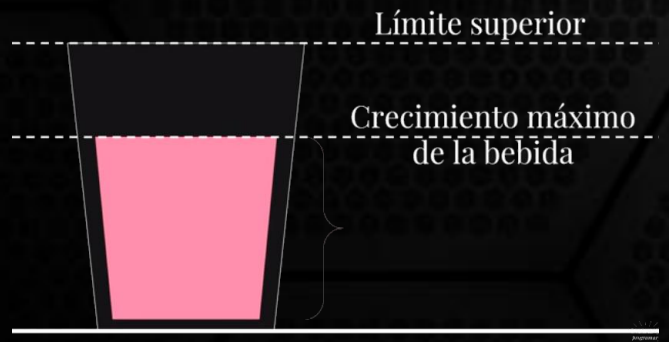
Para evaluar la eficiencia de un algoritmo, no se deben utilizar unidades en tiempo real como microsegundos y nanosegundos. Más bien, se deben utilizar unidades lógicas que expresen una relación entre el tamaño n de un archivo o una matriz y la cantidad de tiempo t necesario para procesar los datos.

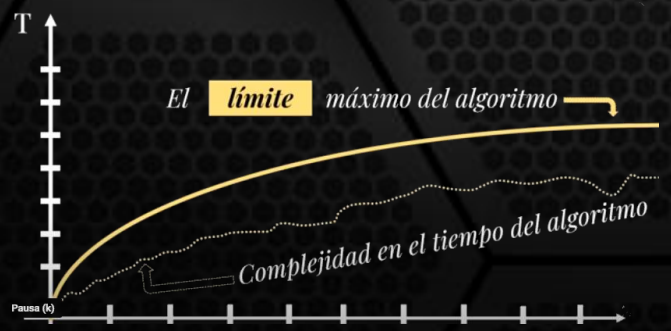
Si existe una relación lineal entre el tamaño n y el tiempo t, es decir, t 1 = cn1, entonces un aumento de los datos en un factor de 5 resulta en el aumento del tiempo de ejecución por el mismo factor; si n2 = 5n1 , entonces t 2 = 5t 1 .

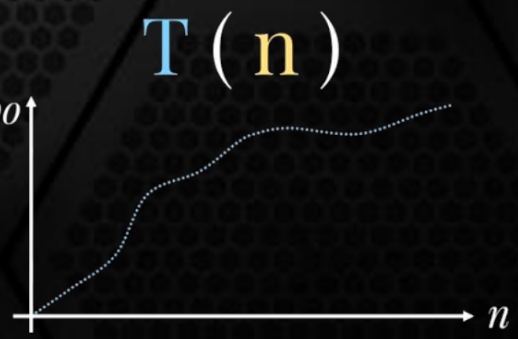
Del mismo modo, si t 1 = log2 n, entonces duplicar n aumenta t en solo una unidad de tiempo. Por lo tanto, si t 2 = log2 (2n), entonces t 2 = t 1 + 1.

Una función que expresa la relación entre n y t suele ser mucho más compleja, y el cálculo de dicha función es importante solo con respecto a grandes cuerpos de datos; Cualquier término que no cambie sustancialmente la magnitud de la función debe ser eliminado de la función. **La función resultante da sólo una medida aproximada de la eficiencia de la función original**. Sin embargo, esta aproximación es lo suficientemente cercana al original, especialmente para una función que procesa grandes cantidades de datos.

Esta medida de eficiencia se **llama complejidad asintótica** y se utiliza cuando se ignoran ciertos términos de una función para expresar la eficiencia de un algoritmo o cuando calcular una función es difícil o imposible y solo se pueden encontrar aproximaciones.





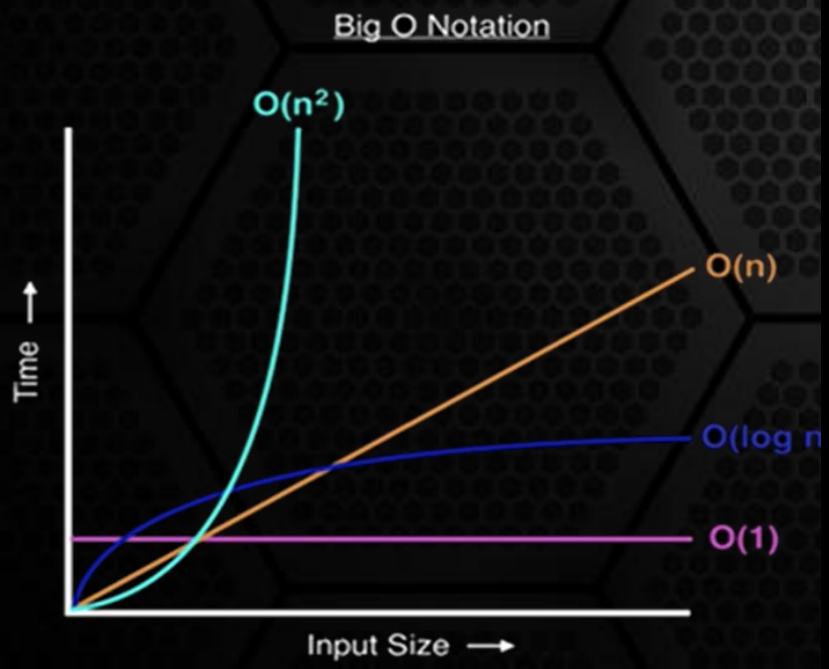




Análisis de la complejidad

Determinado la función que representa al "Comportamiento" del Algoritmo

Para ilustrar el primer caso, considere el siguiente ejemplo: f(n) = n2 + 100n + log10n + 1.000 (2.1)
Para valores pequeños de n, el último término, 1.000, es el más grande. Cuando n es igual a 10, los términos segundo (100n) y último (1.000) están en pie de igualdad con los otros términos, haciendo una pequeña contribución al valor de la función. Cuando n alcanza el valor de 100, el primer y el segundo término hacen la misma contribución al resultado. Pero cuando n se vuelve mayor que 100, la contribución del segundo término se vuelve menos significativa. Por lo tanto, para valores grandes de n, debido al crecimiento cuadrático del primer término (n2), el valor de la función f depende principalmente del valor de este primer término, como lo demuestra la Figura 2.1. Otros términos pueden ser ignorados para n grande.



The growth rate of all terms of function $f(n) = n^2 + 100n + \log_{10}n + 1{,}000$.

n	f(n)	n^2		100n		$\log_{10}n$		1,000	
	Value	Value	%	Value	%	Value	%	Value	%
1	1,101	1	0.1	100	9.1	0	0.0	1,000	90.83
10	2,101	100	4.76	1,000	47.6	1	0.05	1,000	47.60
100	21,002	10,000	47.6	10,000	47.6	2	0.001	1,000	4.76
1,000	1,101,003	1,000,000	90.8	100,000	9.1	3	0.0003	1,000	0.09
10,000	101,001,004	100,000,000	99.0	1,000,000	0.99	4	0.0	1,000	0.001
100,000	10,010,001,005	10,000,000,000	99.9	10,000,000	0.099	5	0.0	1,000	0.00

¿Por qué el análisis de rendimiento?

Hay muchas cosas importantes que deben cuidarse, como la facilidad de uso, la modularidad, la seguridad, la mantenibilidad, etc. **¿Por qué preocuparse por el rendimiento?**
La respuesta a esto es simple, podemos tener todas las cosas anteriores solo si tenemos rendimiento. Así que el rendimiento es como la moneda a través de la cual podemos comprar todas las cosas anteriores. Otra razón para estudiar rendimiento es: ¡la velocidad es divertida!
Para resumir, rendimiento == escala (PERFORMANCE == SCALE). Imagine un editor de texto que puede cargar 1000 páginas, pero puede revisar la ortografía 1 página por minuto o un editor de imágenes que tarda 1 hora en girar su imagen 90 grados a la izquierda o ... lo entiendes. Si una función de software no puede hacer frente a la escala de tareas que los usuarios deben realizar, es tan buena como muerta.



Dados *dos algoritmos para una tarea, ¿cómo averigüemos cuál es mejor?*

Una forma ingenua de hacer esto es: implemente ambos algoritmos y ejecute los dos programas en su computadora para diferentes entradas y vea cuál lleva menos tiempo. Hay muchos problemas con este enfoque para el análisis de algoritmos.
1) Podría ser posible que para algunas entradas, el primer algoritmo funcione mejor que el segundo. Y para algunas entradas el segundo funciona mejor.
2) También podría ser posible que para algunas entradas, el primer algoritmo funcione mejor en una máquina y el segundo funcione mejor en otra máquina para algunas otras entradas.

Como Podemos Analizar el Rendimiento de los Algoritmos?

Aproximación Metodológica por Casos
Notación Asintótica

1. Análisis de algoritmos | Ponderación (Peor, Promedio y Mejores Casos)

Aproximación Metodológica por Casos

Análisis de algoritmos | Conjunto
(Peor, Promedio y Mejores Casos)

El análisis algorítmico busca superar los problemas de una forma ingenua analizando el comportamiento basado en Casos.
Ahora, tomaremos un ejemplo de Búsqueda Lineal y lo analizaremos utilizando el análisis de Casos.
Podemos tener tres casos para analizar un algoritmo:

1) El Peor Caso
2) Caso Promedio
3) Caso Mejor

Consideremos la siguiente implementación de la Búsqueda Lineal.

```cpp
// C++ implementation of the approach
#include <bits/stdc++.h>
using namespace std;

// Linearly search x in arr[].
// If x is present then return the index,
// otherwise return -1
int search(int arr[], int n, int x)
{
    int i;
    for (i = 0; i < n; i++) {
        if (arr[i] == x)
            return i;
    }
    return -1;
}

// Driver Code
int main()
{
    int arr[] = { 1, 10, 30, 15 };
    int x = 30;
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << x << " is present at index "
        << search(arr, n, x);

    getchar();
    return 0;
}
```

SALIDA:

```
30 is present at index 2
```

Análisis de algoritmos | Conjunto
(Peor, Promedio y Mejores Casos)

Análisis del peor de los casos (generalmente realizado)

En el análisis del peor de los casos, calculamos el límite superior en el tiempo de ejecución de un algoritmo. Debemos conocer el caso que provoca que se ejecute un número máximo de operaciones. Para la búsqueda lineal, el peor de los casos ocurre cuando el elemento que se va a buscar (x en el código anterior) no está presente en la matriz. Cuando x no está presente, la función search() lo compara con todos los elementos de arr[] uno por uno. Por lo tanto, la complejidad temporal del peor de los casos de la búsqueda lineal sería Θ(n).

Análisis de caso promedio (a veces hecho)

En el análisis de casos promedio, tomamos todas las entradas posibles y calculamos el tiempo de cómputo para todas las entradas. Suma todos los valores calculados y divide la suma por el número total de entradas. Debemos conocer (o predecir) la distribución de los casos. Para el problema de la búsqueda lineal, supongamos que todos los casos están distribuidos uniformemente (incluido el caso de que x no esté presente en la matriz). Así que sumamos todos los casos y dividimos la suma por (n+1). A continuación se encuentra el valor de la complejidad del tiempo promedio de los casos.

Average Case Time = $\frac{\sum_{i=1}^{n+1} \theta(i)}{(n+1)}$ $\frac{\theta((n+1)*(n+2)/2)}{(n+1)}$

Análisis de algoritmos | Conjunto
(Peor, Promedio y Mejores Casos)

Análisis del mejor caso (falso, final feliz)

En el análisis del mejor de los casos, calculamos el límite inferior en el tiempo de ejecución de un algoritmo. Debemos conocer el caso que provoca que se ejecute un número mínimo de operaciones. En el problema de búsqueda lineal, el mejor de los casos ocurre cuando x está presente en la primera ubicación. El número de operaciones en el mejor de los casos es constante (no depende de n). Así que la complejidad del tiempo en el mejor de los casos sería Θ(1)

Análisis de algoritmos | Conjunto
(Peor, Promedio y Mejores Casos)

Resumen

La mayoría de las veces, hacemos análisis del peor de los casos para analizar algoritmos. En el peor de los casos, garantizamos un límite superior en el tiempo de ejecución de un algoritmo que es buena información.
El análisis de casos promedio no es fácil de hacer en la mayoría de los casos prácticos y rara vez se hace. En el análisis de casos promedio, debemos conocer (o predecir) la distribución matemática de todas las entradas posibles.
El análisis del mejor de los casos es falso. Garantizar un límite inferior en un algoritmo no proporciona ninguna información, ya que en el peor de los casos, un algoritmo puede tardar años en ejecutarse.
Para algunos algoritmos, todos los casos son asintóticamente iguales, es decir, no hay peores y mejores casos. Por ejemplo, Ordenar combinar. Merge Sort realiza operaciones Θ(nLogn) en todos los casos. La mayoría de los otros algoritmos de clasificación tienen los peores y mejores casos. Por ejemplo, en la implementación típica de Quick Sort (donde el pivote se elige como elemento de esquina), lo peor ocurre cuando la matriz de entrada ya está ordenada y lo mejor ocurre cuando los elementos pivotantes siempre dividen la matriz en dos mitades. Para la ordenación de inserción, el peor de los casos se produce cuando la matriz se ordena por inverso y el mejor de los casos se produce cuando la matriz se ordena en el mismo orden que la salida.

Como realizamos la selección del Algoritmo
con una Aproximación Metodológica y Matemático?

Análisis de algoritmos | 2. (Análisis asintótico)



Notación asintótica

$$o \quad O \quad \Theta \quad \Omega \quad \omega$$

El análisis asintótico

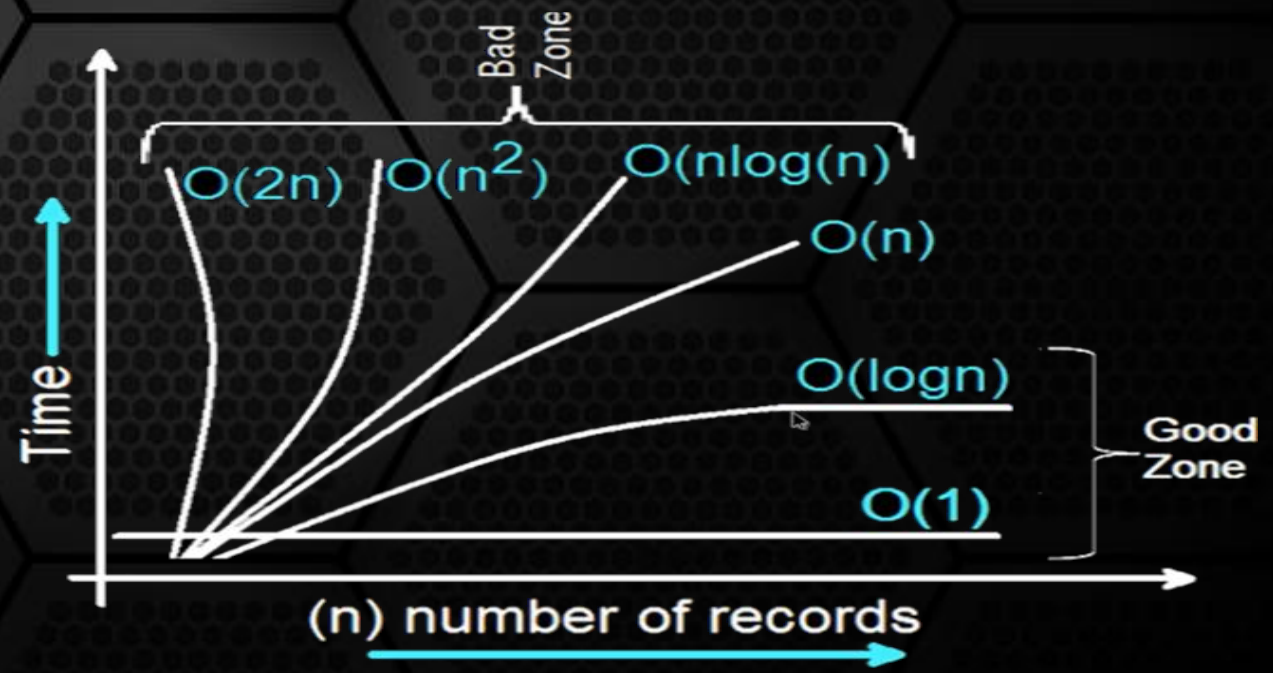
El análisis asintótico es la gran idea que maneja los problemas anteriores en el análisis de algoritmos. En el análisis asintótico, evaluamos el rendimiento de un algoritmo en términos de tamaño de entrada (no medimos el tiempo de ejecución real). Calculamos cómo aumenta el tiempo (o espacio) que tarda un algoritmo con el tamaño de entrada. Por ejemplo, consideremos el problema de búsqueda (búsqueda de un elemento determinado) en una matriz ordenada. Una forma de buscar es la búsqueda lineal (el orden de crecimiento es lineal) y la otra forma es la búsqueda binaria (el orden de crecimiento es logarítmico).

Para entender cómo el Análisis Asintótico resuelve los problemas mencionados anteriormente en el análisis de algoritmos, digamos que ejecutamos la Búsqueda Lineal en una computadora rápida **A** y la Búsqueda Binaria en una computadora lenta **B** y elegimos los valores constantes para las dos computadoras para que nos diga exactamente cuánto tiempo tarda la máquina dada en realizar la búsqueda en segundos. Digamos que la constante para **A** es 0.2 y la constante para **B** es 1000, lo que significa que A es 5000 veces más potente que B. Para valores pequeños de tamaño de matriz de entrada n, la computadora rápida puede tardar menos tiempo. Pero, después de un cierto valor de tamaño de matriz de entrada, la búsqueda binaria definitivamente comenzará a tomar menos tiempo en comparación con la búsqueda lineal a pesar de que la búsqueda binaria se está ejecutando en una máquina lenta. La razón es que el orden de crecimiento de la búsqueda binaria con respecto al tamaño de entrada es logarítmico, mientras que el orden de crecimiento de la búsqueda lineal es lineal. Por lo tanto, *las constantes dependientes de la máquina siempre se pueden ignorar después de un cierto valor de tamaño de entrada*.

El análisis asintótico

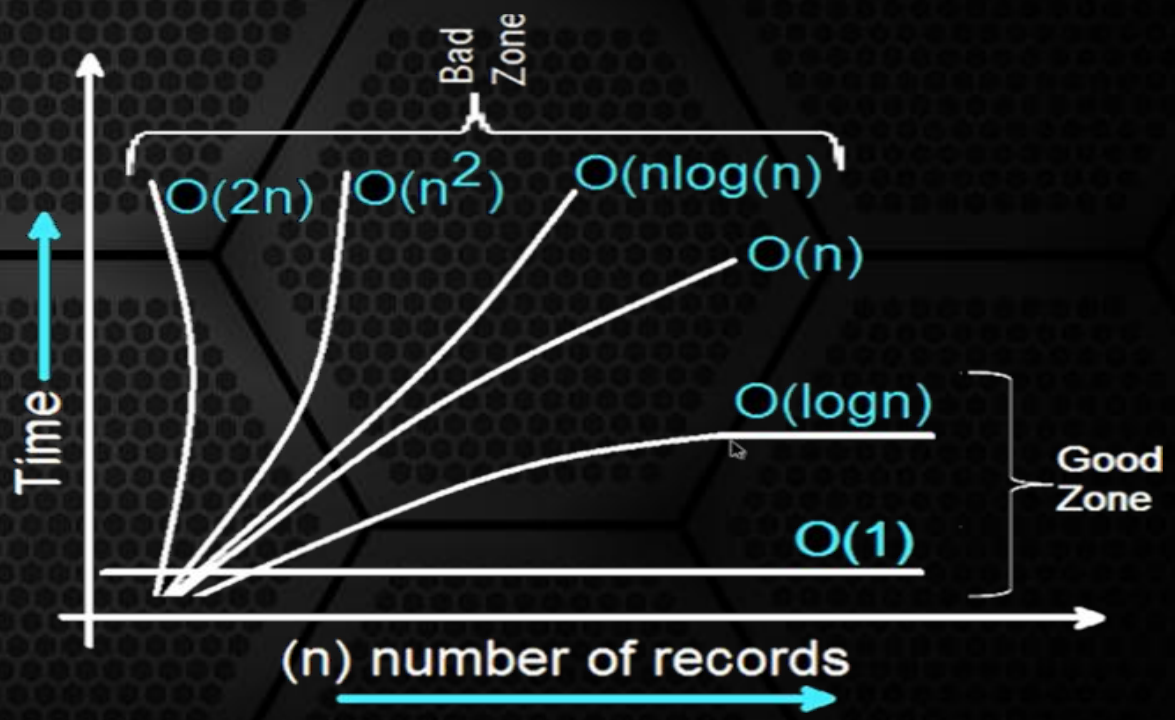
Pero, después de un cierto valor de tamaño de matriz de entrada, la búsqueda binaria definitivamente comenzará a tomar menos tiempo en comparación con la búsqueda lineal a pesar de que la búsqueda binaria se está ejecutando en una máquina lenta. La razón es que ***el orden de crecimiento de la búsqueda binaria con respecto al tamaño de entrada es logarítmico, mientras que el orden de crecimiento de la búsqueda lineal es lineal***. Por lo tanto, las constantes dependientes de la máquina siempre se pueden ignorar después de un cierto valor de tamaño de entrada.
Aquí hay algunos tiempos de ejecución para este ejemplo:
Tiempo de **ejecución de la búsqueda lineal en segundos en A:**0.2 * **n**
Tiempo de **ejecución de la búsqueda binaria en segundos en B:**1000 * **log(n)**

n	Running time on A	Running time on B
10	2 sec	~ 1 h
100	20 sec	~ 1.8 h
10^6	~ 55.5 h	~ 5.5 h
10^9	~ 6.3 years	~ 8.3 h



¿El análisis asintótico siempre funciona?

El análisis asintótico no es perfecto, pero esa es la mejor manera disponible para analizar algoritmos.
Por ejemplo, supongamos que hay dos algoritmos de clasificación que toman 1000nLog(n) y 2nLog(n) tiempo respectivamente en una máquina. *Ambos algoritmos son asintóticamente iguales (el orden de crecimiento es nLog(n). Por lo tanto, con el análisis asintótico, no podemos juzgar cuál es mejor, ya que ignoramos las constantes en el análisis asintótico.*
Además, en el análisis asintótico, siempre hablamos de tamaños de entrada mayores que un valor constante. Es posible que esas grandes entradas nunca se le den a su software y un algoritmo que es asintóticamente más lento, siempre funciona mejor para su situación particular. *Por lo tanto, puede terminar eligiendo un algoritmo que sea asintóticamente más lento pero más rápido para su software.*



Como realizamos la selección del Algoritmo cuando no contamos con un método de Comparación?

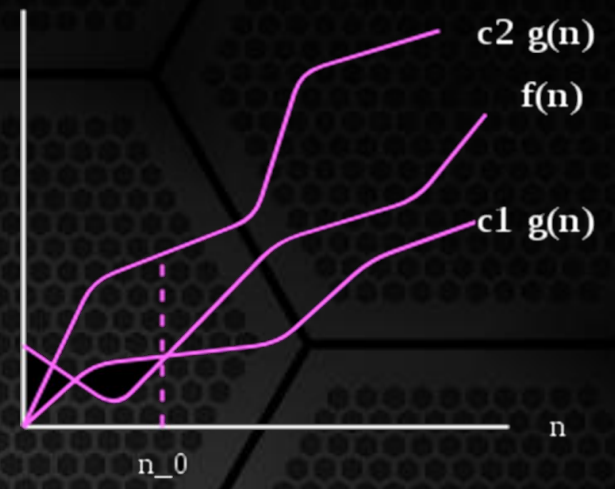
Análisis de algoritmos | 3 (Notaciones asintóticas)

Aproximación Metodológica Matemática.

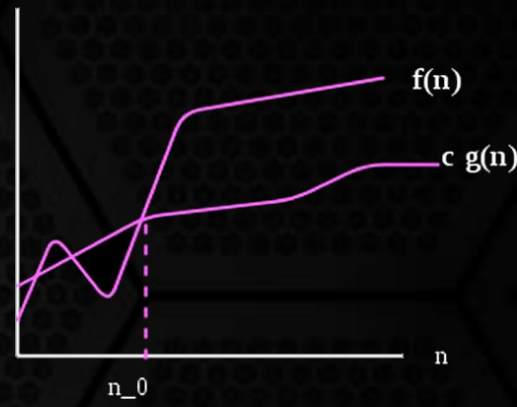


Análisis de algoritmos | Conjunto 3 (Notaciones asintóticas)

Hemos discutido el análisis de los peores, promedio y mejores casos de algoritmos. La idea principal detrás del análisis asintótico es tener una medida de la eficiencia de los algoritmos que no dependen de constantes específicas de la máquina y no requieren que los algoritmos se implementen y los programas tomen el tiempo que tardan en compararse.
Las notaciones asintóticas son herramientas matemáticas para representar la complejidad del tiempo de los algoritmos para el análisis asintótico.



f(n) = theta(g(n))

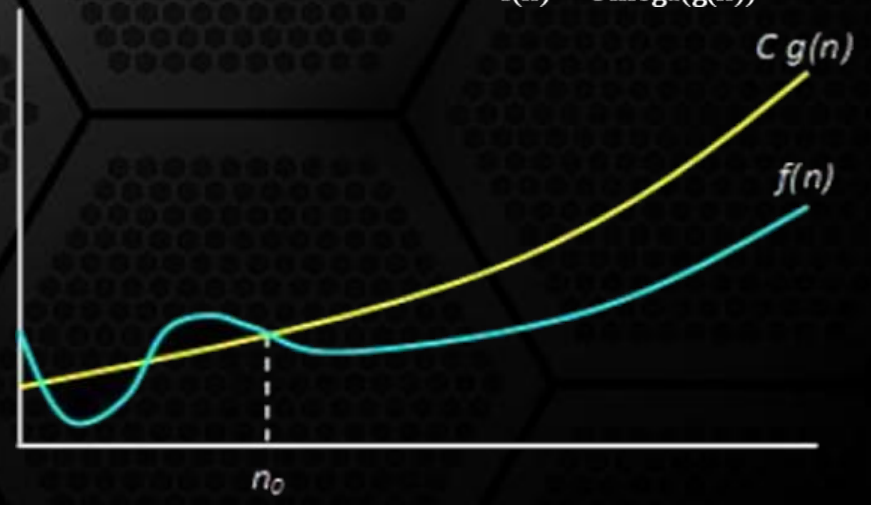


f(n) = Omega(g(n))

Las siguientes 3 notaciones asintóticas se utilizan principalmente para representar la complejidad temporal de los algoritmos.

1) NOTACIÓN Θ
2) NOTACIÓN BIG O
3) NOTACIÓN Ω

¿ΘΩO?



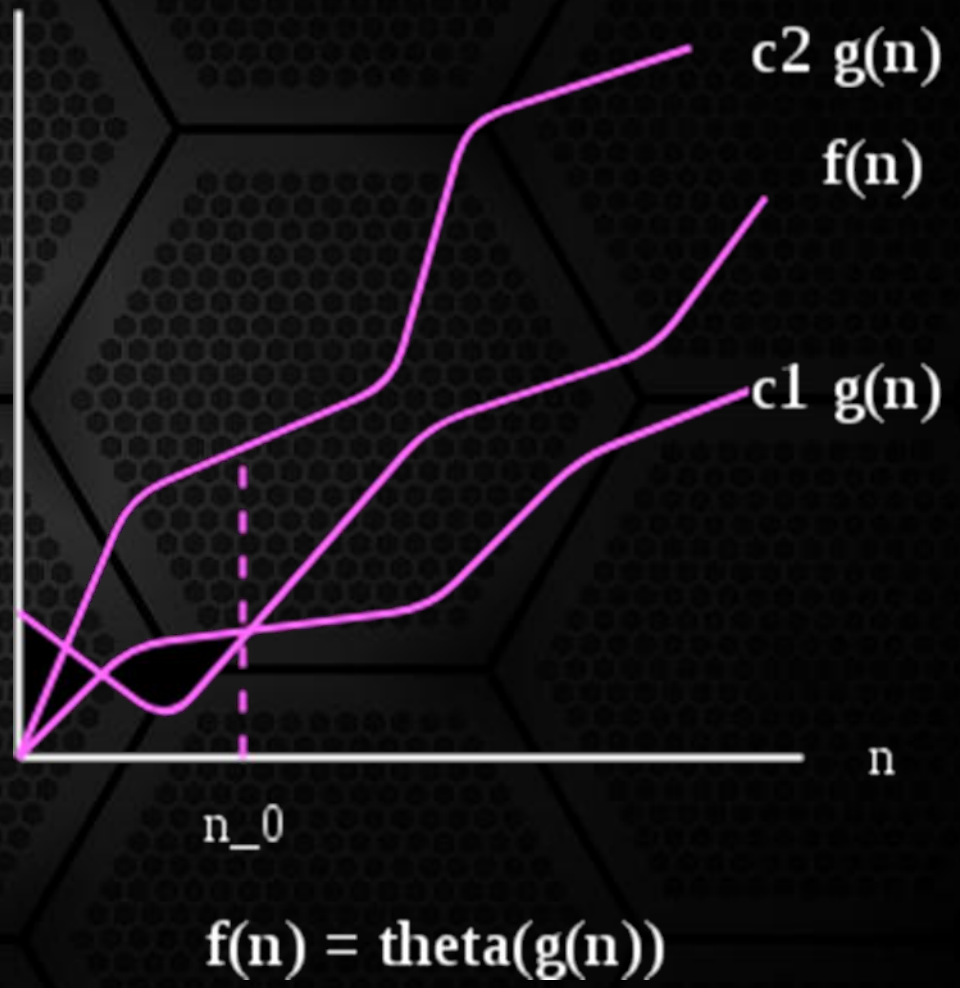
Análisis de algoritmos | Conjunto 3 (Notaciones asintóticas) NOTACIÓN Θ (THETA)

1) Notación Θ (theta):

La notación theta limita una función desde arriba y desde abajo, por lo que define el comportamiento asintótico exacto.

Una forma sencilla de obtener la notación Theta de una expresión es eliminar los términos de orden bajo e ignorar las constantes principales.
Por ejemplo, considere la siguiente expresión.
$3n^3 + 6n^2 + 6000 = \Theta(n^3)$
Eliminar términos de orden inferior siempre está bien porque siempre habrá un número (n) después del cual Θ (n^3) tiene valores más altos que Θ(n^2) independientemente de las constantes involucradas.
Para una función dada g(n), denotamos que Θ(g(n)) es el siguiente conjunto de funciones.

Θ(g(n)) = {f(n): Existen constantes positivas c1, c2 and n0 tales que
0 <= c1*g(n) <= f(n) <= c2*g(n) for all n >= n0}
La definición anterior significa que, si f(n) es theta de g(n), entonces el valor f(n) está siempre entre c1*g(n) y c2*g(n) para valores grandes de n (n >= n0). La definición de theta también requiere que f(n) no sea negativo para valores de n mayor que n0.



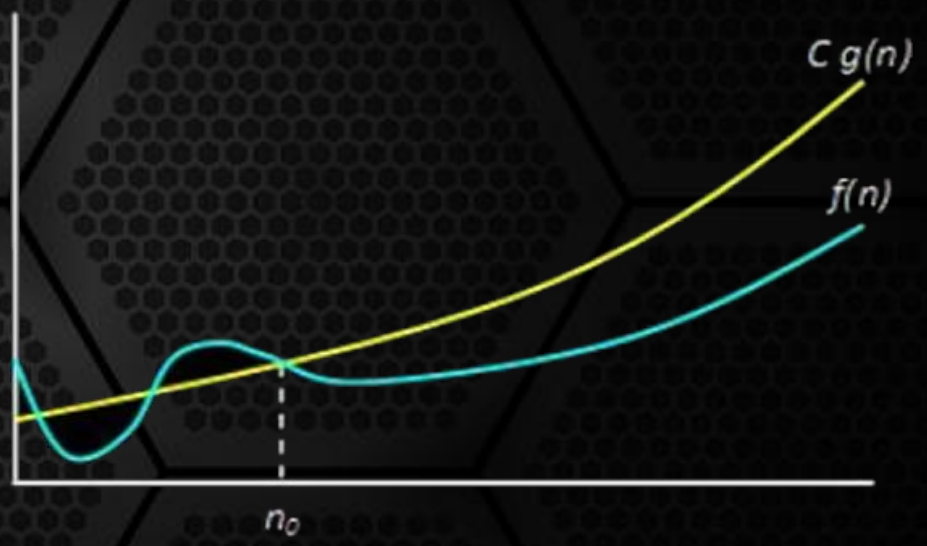
Análisis de algoritmos | Conjunto 3 (Notaciones asintóticas) BIG O

2) Notación Big O:

La notación Big O define un límite superior de un algoritmo, limita una función solo desde arriba. Por ejemplo, considere el caso de Insertion & Sort. Toma tiempo lineal en el mejor de los casos y tiempo cuadrático en el peor de los casos. Podemos decir con seguridad que la complejidad temporal de la ordenación de inserción es O(n^2). Tenga en cuenta que O(n^2) también cubre el tiempo lineal.
Si usamos la notación Θ para representar la complejidad temporal del orden de inserción, tenemos que usar dos instrucciones para los mejores y peores casos:
1. La complejidad de tiempo en el peor de los casos de ordenación de inserción es Θ(n^2).
2. La complejidad de tiempo en el mejor de los casos de Insertion Sort es Θ(n).

La notación Big O es útil cuando solo tenemos un límite superior en la complejidad del tiempo de un algoritmo.
Muchas veces encontramos fácilmente un límite superior simplemente mirando el algoritmo.

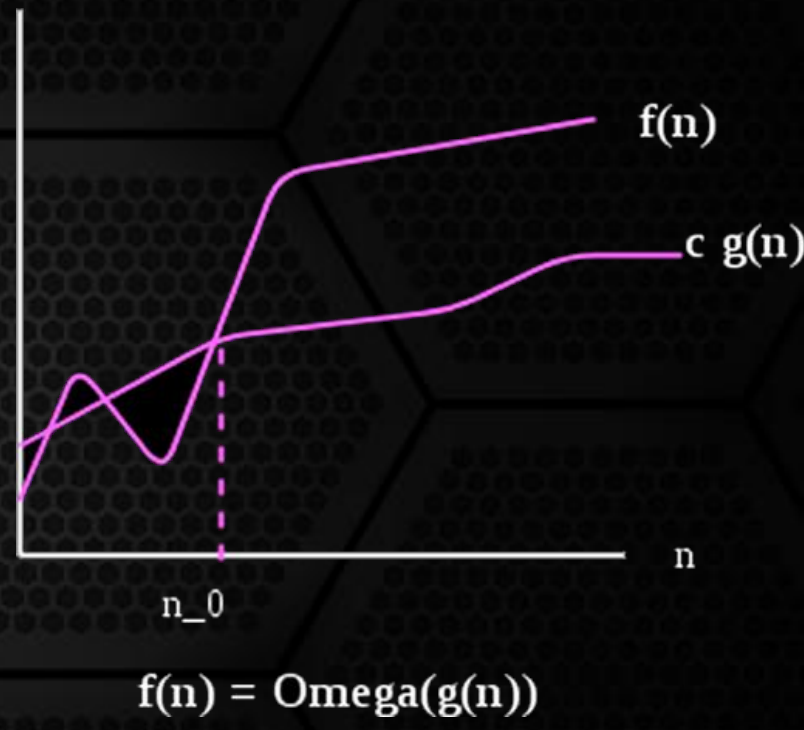
O(g(n)) = { f(n): there exist positive constants c and n0 such that 0 <= f(n) <= c*g(n) for all n >= n0}



Análisis de algoritmos | Conjunto 3 (Notaciones asintóticas) Notación Ω (omega)

3) Notación Ω:

Así como la notación Big O proporciona un límite superior asintótico en una función, la notación Ω proporciona un límite inferior asintótico.
La Ω notación puede ser útil cuando tenemos un límite inferior en la complejidad del tiempo de un algoritmo. Como se discutió previamente, el mejor rendimiento de un algoritmo generalmente no es útil, la notación Omega es la notación menos utilizada entre las tres.
Para una función dada g(n), denotamos por Ω(g(n)) el conjunto de funciones.



Ω (g(n)) = {f(n): existen contantes positivas c y n0
tales que 0 <= c*g(n) <= f(n) for all n >= n0}.

Consideremos el mismo ejemplo de ordenación de inserción aquí. La complejidad temporal de “Insertion Sort “ se puede escribir como Ω(n), pero no es una información muy útil sobre el orden de inserción, ya que generalmente nos interesa el peor de los casos y, a veces, en el caso promedio.

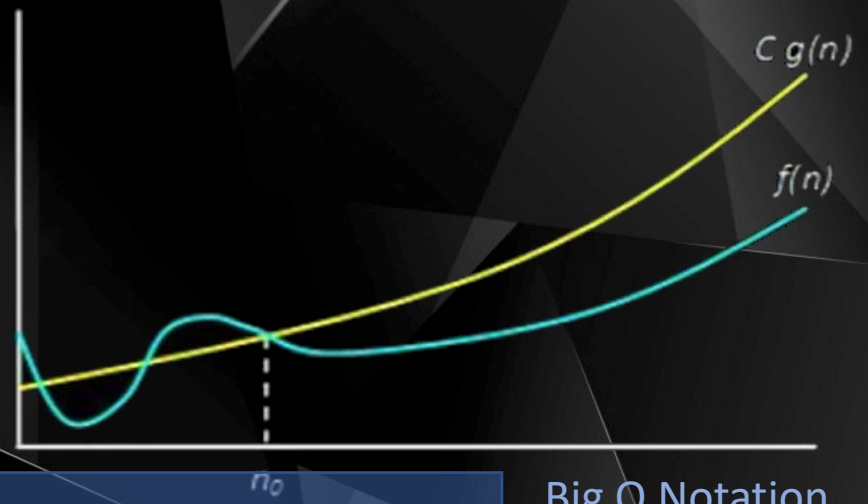
Coffee Break

Fin parte I



Volvemos en 15 Minutos...

Big-O



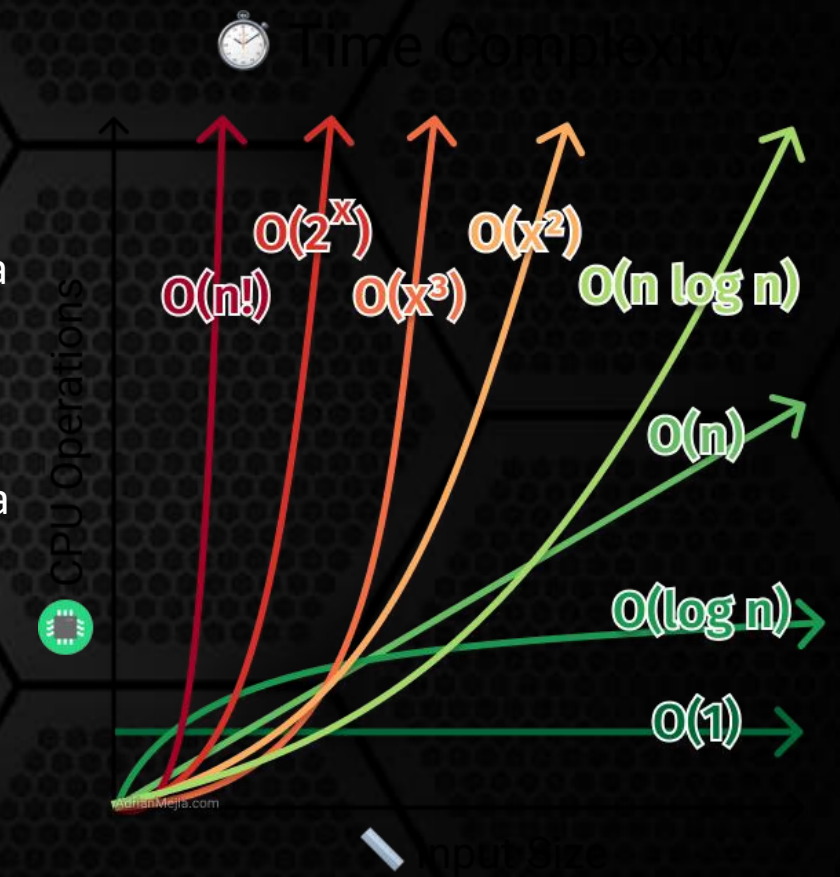
Big O Notation

Notación Big O

¿Cómo calcular la complejidad del tiempo de cualquier algoritmo o programa?
La métrica más común es usar la notación Big O.
Aquí hay algunos aspectos destacados sobre **Big O Notation**:

•La notación Big O es un marco para analizar y comparar algoritmos.
•Cantidad de trabajo que la CPU tiene que hacer (complejidad de tiempo) a medida que el tamaño de entrada crece (hacia el infinito).
•Big O = Función Big Order. Elimine constantes y términos de orden inferior. P. ej..
Se convierte. O(3*n^2 + 10n + 10) -> O(n^2)
•La notación Big O se preocupa por el peor de los casos. Por ejemplo, cuando desea ordenar y los elementos de la matriz están en orden inverso para algunos algoritmos de clasificación.

Por ejemplo, si tiene una función que toma una matriz como entrada, si aumenta el número de elementos de la colección, seguirá realizando las mismas operaciones; tiene un tiempo de ejecución constante. Por otro lado, si el trabajo de la CPU crece proporcionalmente al tamaño de la matriz de entrada, tiene un tiempo de ejecución lineal.O(n)



Notación Big O

Introducción

La notación Big O, es decir, básicamente expresar la complejidad tiempo/espacio de un algoritmo en términos de Big O viene en el papel cuando desea encontrar el tiempo / espacio consumido por su algoritmo.
Porque todos sabemos una cosa que encontrar una solución a un problema no es suficiente, pero resolver ese problema en el mínimo tiempo / espacio posible también es necesario.

Aunque no le importa demasiado cómo se está desempeñando su algoritmo o tomando tiempo / espacio a menos que esté trabajando en un entorno competitivo, proyectos de la vida real, haciendo programación competitiva porque estos son los lugares donde se le exigirá no solo encontrar la solución sino también optimizarla siempre que sea posible en términos de espacio / tiempo.

Si hablamos de programación competitiva, entonces quieres que tu algoritmo sea más rápido que otros, de lo contrario no podrás rendir mejor que otros. Por otro lado, cuando trabajas en proyectos de la vida real o como desarrollador de software, entonces quieres que el tiempo de carga de tu aplicación u otras tareas se ejecuten rápidamente incluso en términos de complejidad espacial.

*En otras palabras, se puede decir que está en el camino correcto para aprender estructuras de datos y algoritmos o sus habilidades de desarrollo cuando comienza a observar la complejidad de su algoritmo. Pero a partir de todo esto llegamos a saber cuál es la complejidad y por qué la estamos estudiando, pero qué es Big O, **Big O establece un peor tiempo de ejecución.***

Notación Big O

QU SABEMOS DE LA B SQUEDA BINARIA DE LA IMAGEN MUESTRA ES M S R PIDA, VERDAD? NO, ES MUCHO M S QUE ESO CUANDO EL NO. DE JUGUETES AUMENTA. SI HAY 1.000 MILLONES DE JUGUETES Y SE NECESITAN 30 MS (LOG2 1.000.000.000 ES CASI 30), ENTONCES SER A M S COMO 33 MILLONES DE VECES M S R PIDO.

UNA COSA QUE LLEGAMOS A SABER DE LA B SQUEDA LINEAL FRENTE A LA B SQUEDA BINARIA ES C MO EL TIEMPO DE EJECUCI N DEL ALGORITMO AUMENTA COMO EL NO. DE JUGUETES (ART CULOS DE LA LISTA) AUMENTA.
AH ES DONDE BIG O ENTRA EN JUEGO.

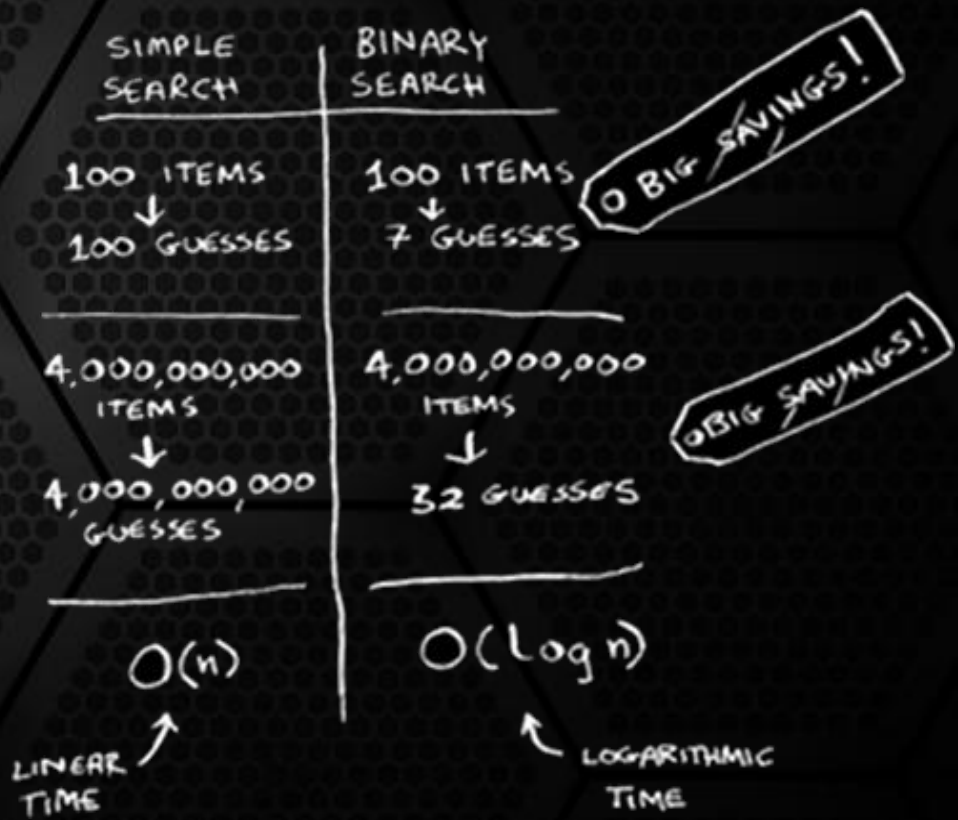
->BIG O TE DICE QU TAN R PIDO ES TU ALGORITMO.

CONSIDERE UN CASO EN EL QUE TENGA UNA LISTA DE TAMA O N.
LA B SQUEDA SIMPLE REQUIERE VERIFICAR CADA ELEMENTO PARA QUE TOME N OPERACIONES. POR LO QUE EL TIEMPO DE EJECUCI N EN NOTACI N BIG O ES O(N).

D NDE EST N LOS SEGUNDOS?
NO HAY NINGUNO: BIG O NO TE DICE LA VELOCIDAD DEL ALGORITMO EN SEGUNDOS.

BIG O LE PERMITE COMPARAR EL N MERO DE OPERACIONES.

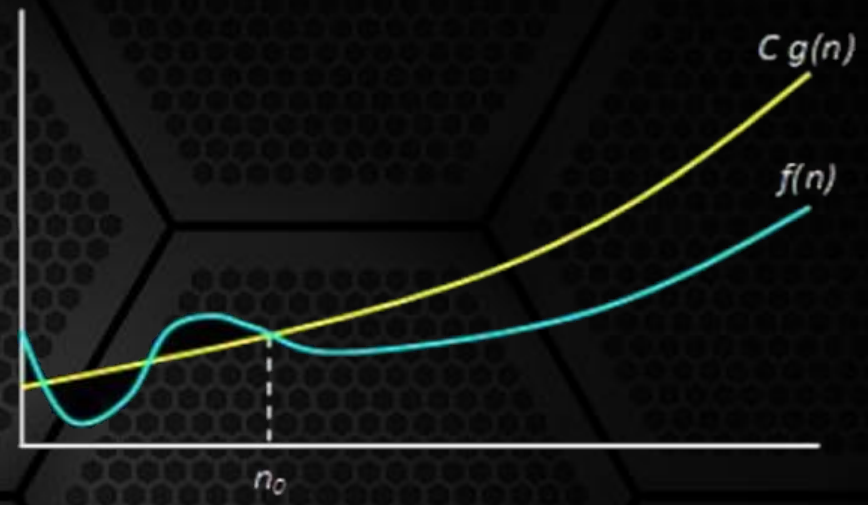
Ejemplo: Búsqueda de Juguetes.



¿Cómo compararás los algoritmos?

Pongamos un ejemplo. Para resolver el mismo problema, tiene dos funciones:

```
f(n) = 4n^2 +2n+4 and g(n) =4n+4
Para f(n) =4n^2+2n+4
Entonces
f(1)= 4+2+4
f(2)=16+4+4
f(3)=36+6+4
f(4)=64+8+4
```



COMO SE PUEDE VER AQU LA CONTRIBUCI N DE N 2 ES UNA FUNCI N CUADR TICA CUYO VALOR AUMENTAR CON EL AUMENTO DEL VALOR DE N. ENTONCES, CUANDO SU VALOR DE N SE VUELVE MUY GRANDE, LA CONTRIBUCI N DE N 2 SER DEL 99% DEL VALOR EN F(N).
POR LO TANTO, AQU LOS T RMINOS DE ORDEN BAJOS SE PUEDEN IGNORAR, YA QUE SON RELATIVAMENTE INSIGNIFICANTES COMO SE DESCRIBI ANTERIORMENTE.

EN ESTE F(N), PODEMOS IGNORAR 2N Y 4 EN COMPARACI N CON N 2.

n^2+2n+4 —> n^2. Aquí n^2 es el de Mayor Crecimineto. Para g(n) =4n+4 aqui
g(1)=4+4 g(2)=8+4 g(3)=12+4 g(4)=16+4

Punto a tener en cuenta:

NO ESTAMOS CONSIDERANDO T RMINOS QUE EST N CRECIENDO LENTAMENTE Y NO TIENEN MUCHO EFECTO EN COMPARACI N CON OTROS T RMINOS Y MANTENEMOS UNO QUE CRECE M S R PIDO.

COMPLEJIDAD DEL TIEMPO:

LA COMPLEJIDAD DEL TIEMPO ES UNA FORMA DE REPRESENTAR O CONOCER C MO EL TIEMPO DE EJECUCI N DE UNA FUNCI N AUMENTA / DISMINUYE A MEDIDA QUE AUMENTA / DISMINUYE EL TAMA O DE LA ENTRADA.
HAY MUCHOS TIPOS DE COMPLEJIDAD DEL TIEMPO, POR EJEMPLO:

- TIEMPO LINEAL —-> YA DISCUTIDO EN EL ESCENARIO ANTERIOR DONDE EL TIEMPO AUMENTA LIEALMENTE.

- TIEMPO CONSTANTE —-> AQU EL TIEMPO DE EJECUCI N DE LA FUNCI N PERMANECE CONSTANTE AUNQUE AUMENTE EL VALOR DE N A UNA VELOCIDAD MUY ALTA.

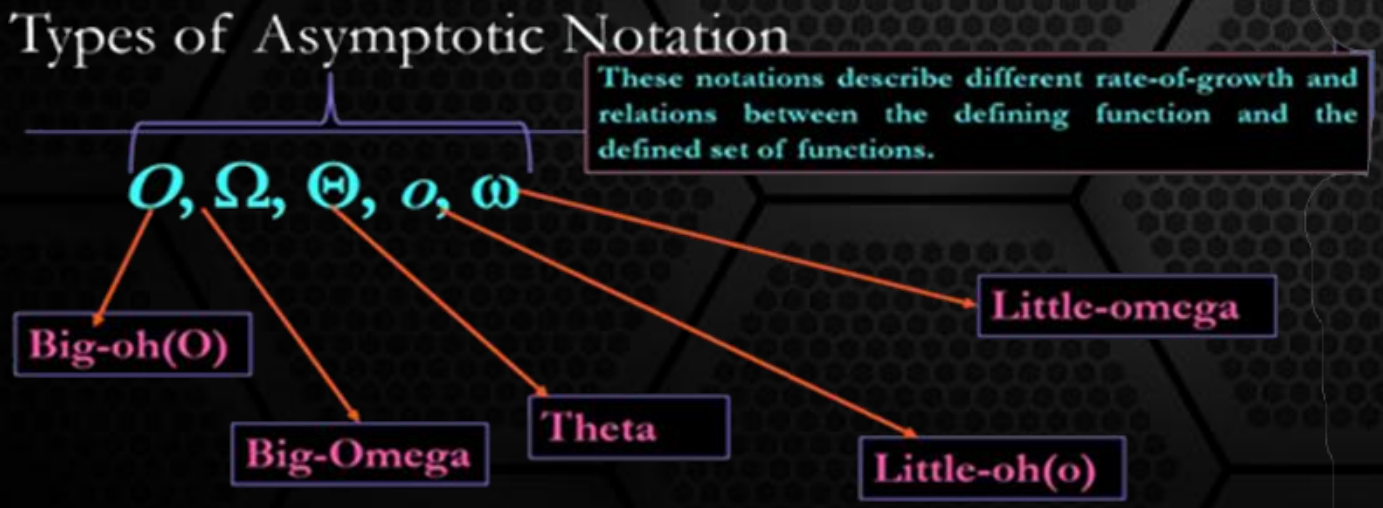
- TIEMPO CUADR TICO —-> AQU EL TIEMPO DE EJECUCI N DE LAS FUNCIONES AUMENTA DE FORMA EXPONENCIAL AL AUMENTAR EL VALOR DE N.

Notación Asintótica

•LAS NOTACIONES ASINT TICAS SON B SICAMENTE NOTACIONES MATEM TICAS QUE SON TILES PARA REPRESENTAR LA COMPLEJIDAD DEL TIEMPO DE EJECUCI N DEL ALGORITMO.

•AYUDA A ANALIZAR EL TIEMPO DE EJECUCI N DE UN PROGRAMA EN FUNCI N DEL TAMA O DE LA ENTRADA DADA.

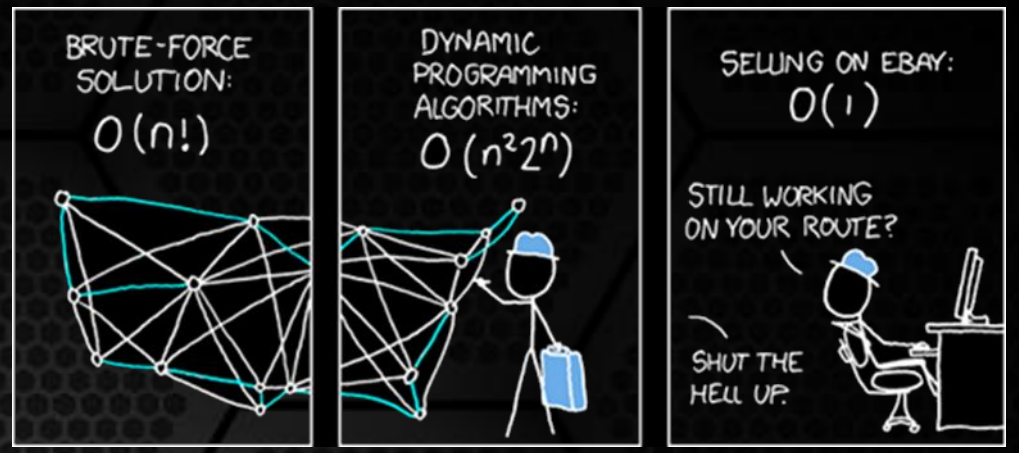
•HAY TRES TIPOS DE NOTACIONES ASINT TICAS UTILIZADAS EN LA COMPLEJIDAD DEL TIEMPO.
COMO SE MUESTRA A CONTINUACI N:



PERO COMO NUESTRA PRINCIPAL PREOCUPACI N ES ENTENDER

BIG O – L MITE SUPERIOR: SI F(N) <= C*G(N) PARA TODO N > NO
ENTONCES F(N) = O(G(N))
AYUDA A DESCRIBIR EL RENDIMIENTO O LA COMPLEJIDAD DE NUESTRO ALGORITMO.
BIG O DETERMINA EL PEOR DE LOS CASOS, ES DECIR, LA MAYOR CANTIDAD DE TIEMPO NECESARIO EN LA EJECUCI N DEL PROGRAMA.
F(N) = O(G(N)), MUESTRA CLARAMENTE QUE HAY CONSTANTES POSITIVAS C Y NO, TALES QUE 0 ≤ F(N) ≤ CG(N) PARA TODO N ≥ NO.
LOS VALORES DE C Y NO SON INDEPENDIENTES DE N.
EJEMPLO: ESCRIBIR EN FORMA DE F(N)<=C*G(N) CON F(N)=4N+3 Y G(N)=5N
CUANDO NO = 3, LA CONDICI N ANTERIOR SE VUELVE VERDADERA, ES DECIR, 4N+3<=5N PARA NO=3 Y C=5.

Notaciones asintóticas comunes:



Si no entiendes el meme, entonces probablemente quieras leer sobre el muy famoso problema Travelling Salesman (Programación Dinámica). Aquí hay cinco tiempos de ejecución de Big O que encontrará mucho, ordenados de más rápido a más lento:

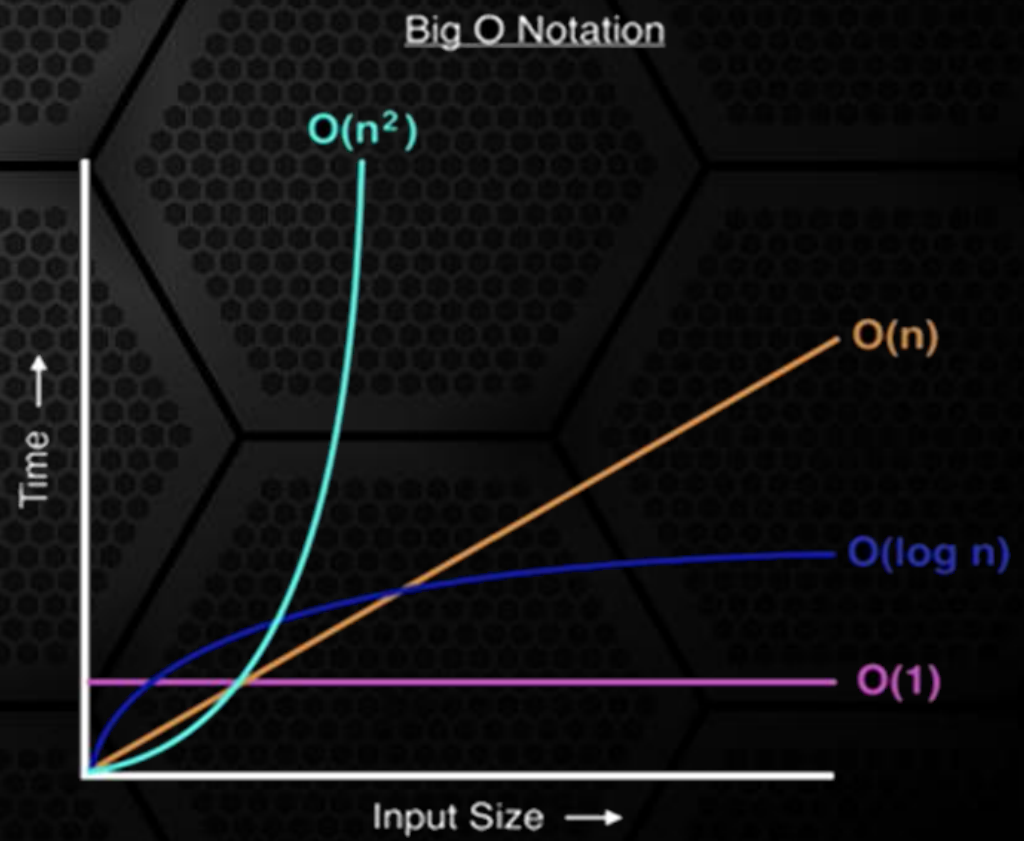
•O(log n), también conocido como log time.
Ejemplo: Búsqueda binaria.

•O(n), también conocido como tiempo lineal.
Ejemplo: Búsqueda simple.

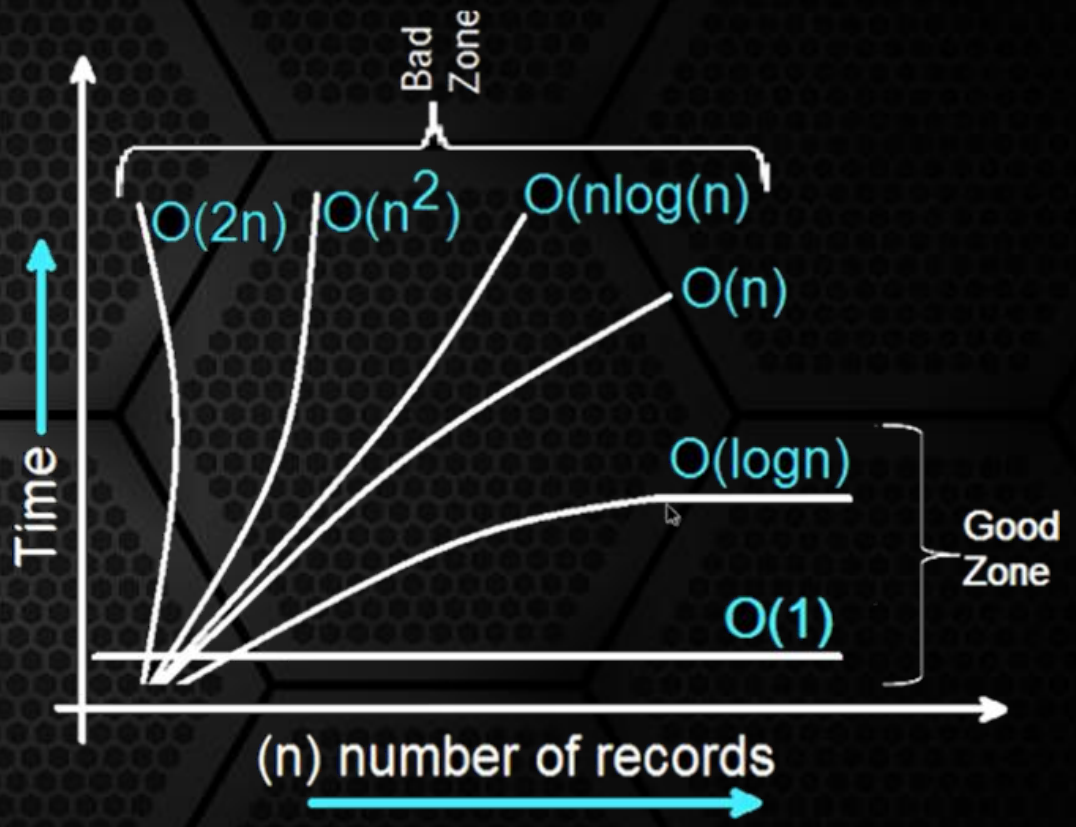
•O(n * log n). Ejemplo: Un algoritmo de clasificación rápida, como quicksort.

•O(n2). Ejemplo: Un algoritmo de clasificación lenta, como la ordenación por selección.

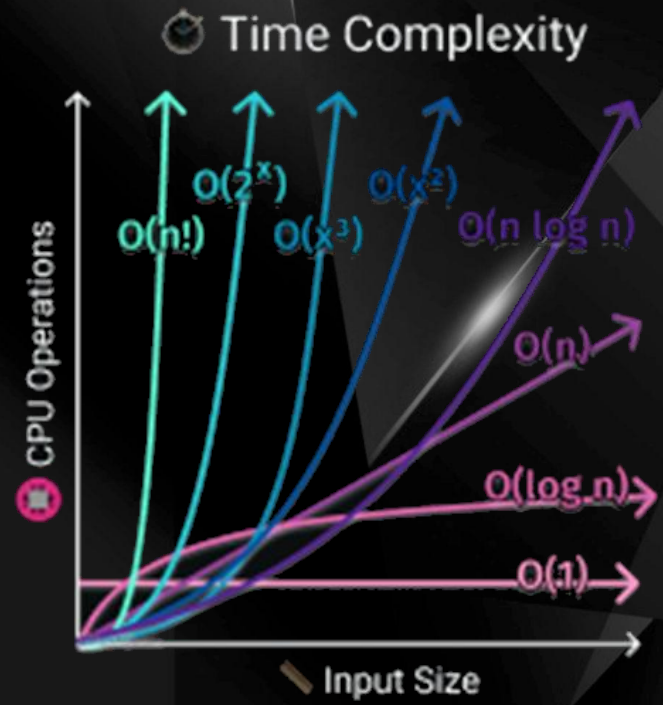
•¡O(n!). Ejemplo: Un algoritmo realmente lento, como el vendedor ambulante.



¿Cómo encontrar la complejidad del tiempo y el espacio de un algoritmo?

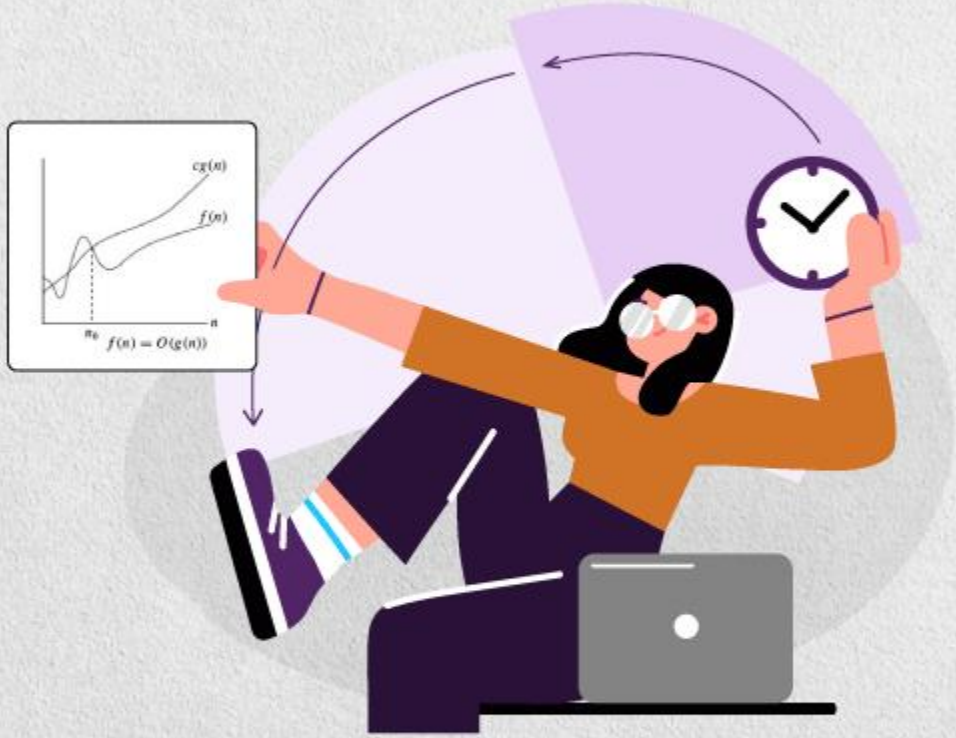






How to find time complexity of algorithms?

From Code to Big O notation



Explained: Time & Space complexity

[Complejidad del tiempo y el espacio en algoritmos | Coding Ninjas Blog]

La Complejidad del tiempo y el espacio -Introducción

Innumerables veces, hemos escuchado a nuestros profesores de matemáticas especificar diferentes métodos para resolver un problema en particular. El objetivo es utilizar técnicas cortas y fáciles en lugar de técnicas largas y complicadas.
Del mismo modo, al escribir código, es una buena práctica utilizar recursos mínimos para la ejecución. Aquí, los recursos indican los requisitos de tiempo y memoria de un algoritmo.
Imagina que buscaste algo en Google y tardaste más de una hora en mostrar el resultado. ¿Volverías a usar la búsqueda de Google? Probablemente no.
El algoritmo de búsqueda de Google es solo un ejemplo. Los algoritmos aparecen por todas partes, prácticamente en cualquier lugar donde haya una computadora involucrada.
Por lo tanto, las empresas contratan programadores calificados que pueden escribir códigos eficientes porque comprender el rendimiento de un algoritmo puede ayudar a la empresa a escalar más allá de sus expectativas más salvajes del negocio.

Este apartado discutirá la definición primaria de complejidad de tiempo y espacio, seguida de varios ejemplos para demostrar cómo evaluarlos.

1. Complejidad del Tiempo
2. Complejidad del Espacio

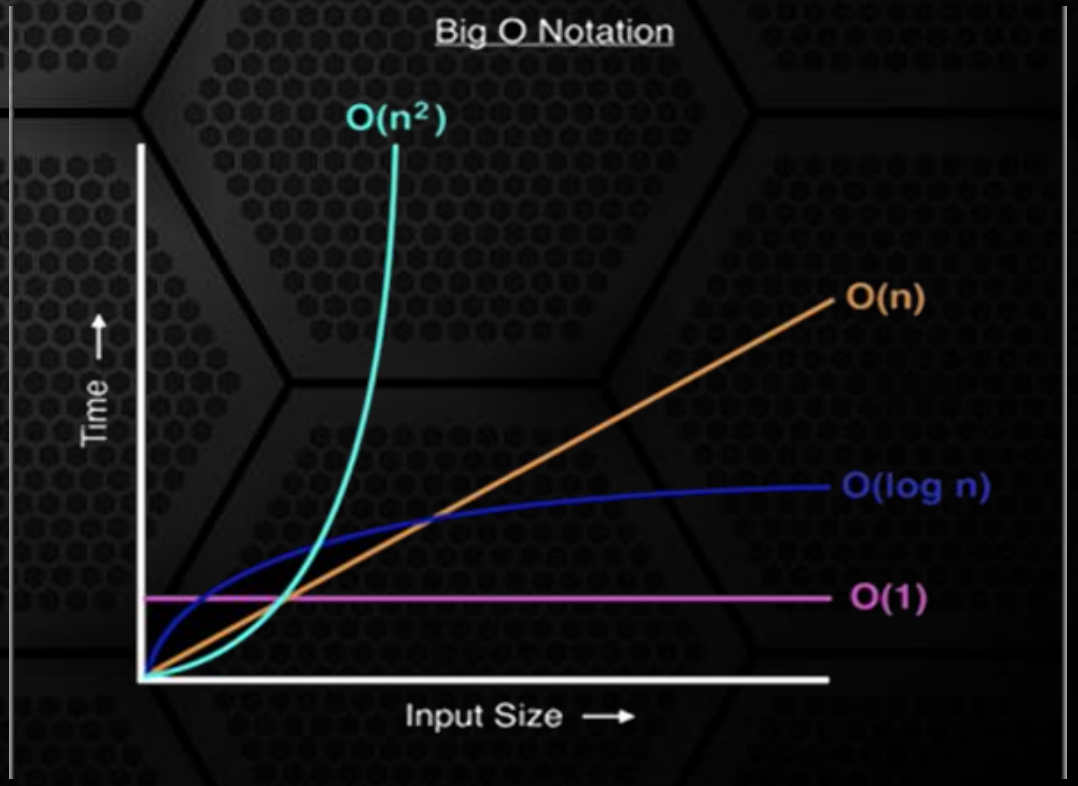
1. ¿Qué es la complejidad del tiempo?

Como Calcular la Complejidad?

¿Qué es la complejidad del tiempo?

Para recapitular **la complejidad del tiempo,** se estima cómo funciona un algoritmo, independientemente del tipo de máquina en la que se ejecute. Puede obtener la complejidad del tiempo "contando" el número de operaciones realizadas por su código. Esta complejidad de tiempo se define como una función del tamaño de entrada utilizando la notación Big-O. indica el tamaño de entrada, mientras que O es la función de tasa de crecimiento del peor de los casos. n

Utilizamos la notación **Big-O** para clasificar los algoritmos en función de su tiempo de ejecución o espacio (memoria utilizada) a medida que crece la entrada. La función es la tasa de crecimiento en función del tamaño de entrada.



¿Qué es la complejidad del tiempo?

En los dos códigos anteriores:
•El código 1 imprime Hello! una vez y detiene la ejecución.
•Por el contrario, el código 2 está imprimiendo Hello! 5 veces, o puede decir N veces, donde N es el número ingresado por el usuario y luego detiene la ejecución.
•Cuando los ejecute con un compilador de C ++, verá la oración "*El código se ejecuta en 0.2 segundos ".* Muchos de nosotros pensamos que esto es una complejidad de tiempo, pero este no es el caso.

En cambio, podemos inferir del ejemplo anterior que el tiempo requerido para completar el programa es igual a la suma del número de veces que se ejecuta cada instrucción.

Más precisamente, ***la complejidad del tiempo se refiere a la cantidad de tiempo requerido por el algoritmo o programa para ejecutarse hasta su finalización.***

La complejidad del tiempo debe representarse matemáticamente, utilizando notaciones asintóticas como Big O, Big Θ y Big Ω.

Code 1

```
#include <iostream>
using namespace std;

int main()
{
cout<<"Hello!\n";
return 0;
}

OUTPUT
Hello!
```

Code 2

```
int main()
{
int N;
cout<<"Enter the value of N: ";
cin>>N;
for(int i=0;i<N;i++)
{
   cout<<"Hello!\n";
}
return 0;
}

OUTPUT (For input N=5)
Hello!
Hello!
Hello!
Hello!
Hello!
```

¿Cómo Calcular y encontrar la complejidad del tiempo?

Los pasos involucrados en encontrar la complejidad de tiempo de un algoritmo son:

1. Encuentre el número de sentencias con complejidad de tiempo constante (O(1)).
2. Encuentre el número de instrucciones con órdenes de complejidad más altos como O(N), $O(N^2)$, O(log N), etc.
3. Exprese la complejidad del tiempo total como una suma de la constante.
4. Elimine los términos no dominantes: cualquier cosa que represente como $O(N)^2+N)$ se puede escribir como $O(N^2)$ para obtener una complejidad de tiempo de orden superior.
5. Ignorar constantes; si lo hay, no afecta a la complejidad general del tiempo.
6.

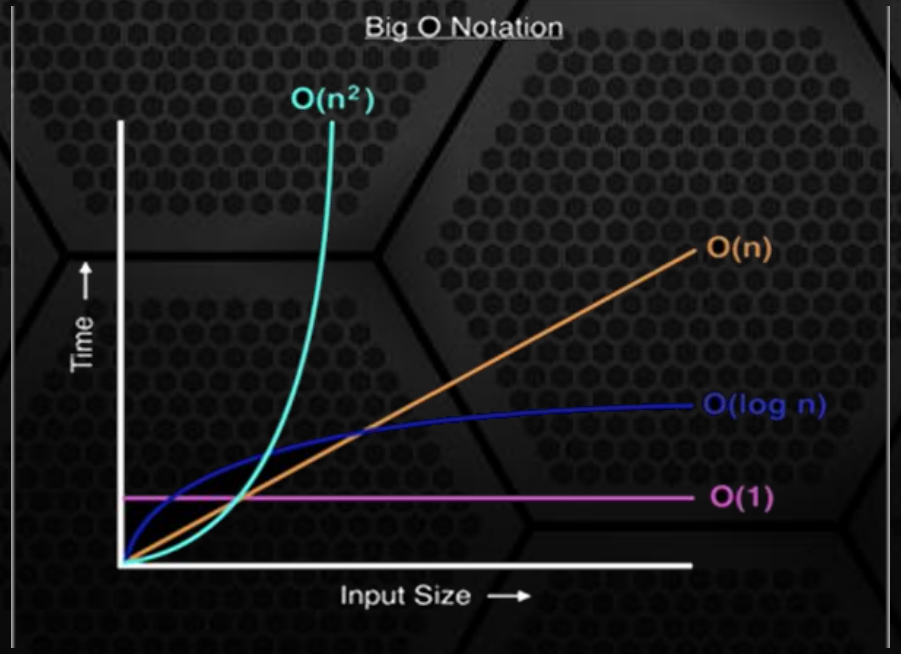
Por ejemplo, cualquier cosa que puedas pensar que es O(3n) es O(n).
Ahora que sabemos cómo encontrar la complejidad de tiempo del algoritmo, veamos algunos ejemplos de complejidad de tiempo para comprender mejor la evaluación de complejidad de tiempo.

Veamos como Calcularlo…

Complejidad O(1)
Ejemplos de cálculo de complejidad de tiempo

- SOLO HAY INICIALIZACI N, INSTRUCCIONES DE ENTRADA Y SALIDA, UNA OPERACI N ARITM TICA Y UNA INSTRUCCI N RETURN EN EL C DIGO ANTERIOR.
TODOS ESTOS TIENEN UNA COMPLEJIDAD TEMPORAL O(1).

- AS QUE LA COMPLEJIDAD TEMPORAL DE TODO EL C DIGO ES O(1).




```cpp
#include <iostream>
using namespace std;

#include <iostream>
using namespace std;

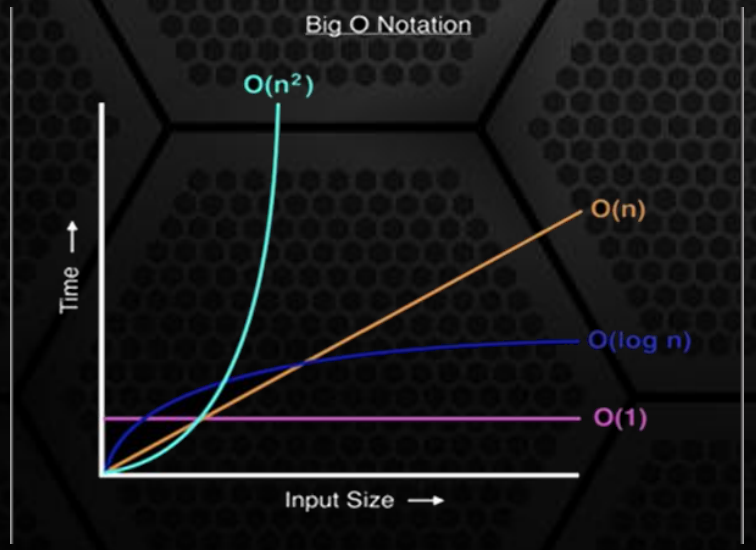
int main()
{
int a,b,c;
cout<<"Enter two numbers: ";
cin>>a>>b;
c=a+b;
cout<<a<<" + "<<b<<" = "<<c;
return 0;
}
```

SALIDA:

INTRODUZCA DOS N MEROS: 2 + 3 = 5

Complejidad O(N)
Ejemplos de cálculo de complejidad de tiempo

- EN EL SIGUIENTE C DIGO, HAY UN BUCLE FOR ADEM S DE LA INICIALIZACI N, INSTRUCCIONES DE ENTRADA Y SALIDA, UNA OPERACI N ARITM TICA Y UNA INSTRUCCI N RETURN.
- APARTE DEL BUCLE, EL RESTO DE LAS L NEAS TIENEN UNA COMPLEJIDAD TEMPORAL O(1).
- EL BUCLE FOR SE EJECUTAR N VECES, POR LO QUE SU COMPLEJIDAD DE TIEMPO SER O(N).
- AS , TODO EL C DIGO TENDR UNA COMPLEJIDAD TEMPORAL DE O(N + 9), PERO LA CONSTANTE SUELE DESCUIDARSE PARA VALORES GRANDES DE N, Y LA COMPLEJIDAD TEMPORAL SER O(N), TAMBI N CONOCIDA COMO TIEMPO LINEAL.

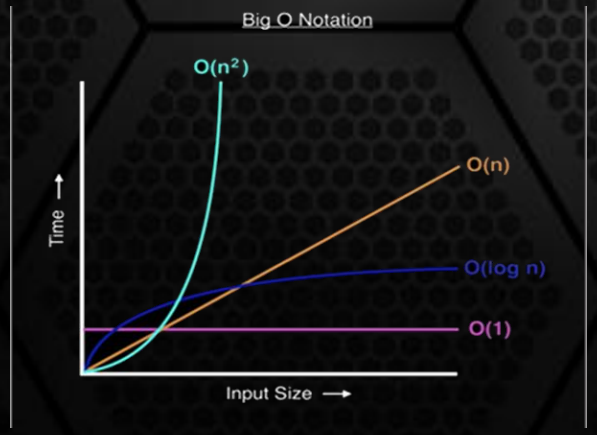



```
Enter the length of the array: 5
Enter the element in position 0: 1
Enter the element in position 1: 2
Enter the element in position 2: 3
Enter the element in position 3: 4
Enter the element in position 4: 5
The sum of the elements is 15
```

```
int main()
{
    int N,sum=0;
    cout<<"Enter the length of the array: ";
    cin>>N;
    int arr[N];
    for(int i=0;i<N;i++) //time complexity O(n)
    {
    cout<<"Enter the element in position "<<i<<": ";
    cin>>arr[i];
    sum+=arr[i];
    }
cout<<"The sum of the elements is "<<sum;
return 0;
}
```

Complejidad $O(N^2)$
Ejemplos de cálculo de complejidad de tiempo

- EN LOS DOS EJEMPLOS ANTERIORES, YA DISCUTIMOS LA COMPLEJIDAD TEMPORAL DE FOR Y OTRAS DECLARACIONES.
- PERO AQU , LOS BUCLES EST N ANIDADOS, LO QUE IMPLICA QUE EL BUCLE INTERNO EJECUTAR N VECES PARA UNA ITERACI N DEL BUCLE EXTERNO.
- POR LO TANTO EJECUTAR N × N VECES (N^2), POR LO QUE SU COMPLEJIDAD TEMPORAL SER $O(N^2)$. POR LO TANTO, TODO EL C DIGO TENDR UNA COMPLEJIDAD DE TIEMPO DE $O(N^2 + 9)$, DONDE 9 ES LA SUMA DE LA COMPLEJIDAD TEMPORAL O(1) DE LAS DECLARACIONES RESTANTES.
- COMO YA SABEMOS, PARA VALORES GRANDES DE N, LA CONSTANTE GENERALMENTE SE DESCUIDA, Y POR LO TANTO LA COMPLEJIDAD TEMPORAL SER $O(N^2)$.
- ORDENACI N DE SELECCI N, ORDENACI N DE INSERCI N SON ALGUNOS EJEMPLOS DE $O(N^2)$ COMPLEJIDAD DEL TIEMPO.




```
#include <iostream>
using namespace std;

#include <iostream>
using namespace std;

int main()
{
int N,sum=0;
cout<<"Enter the dimensions of the square matrix: ";
cin>>N;
int arr[N][N];
for(int i=0;i<N;i++) //outer loop
{
    for(int j=0;j<N;j++) //inner loop
    {
        cout<<"Enter the element in position   "<<i<<","<<j<<": ";
        cin>>arr[i][j];
        sum+=arr[i][j];
    }
}
cout<<"The sum of the elements is "<<sum;
return 0;
}
```

Output:

```
Enter the dimensions of the square matrix: 2
Enter the element in position 0,0: 1
Enter the element in position 0,1: 2
Enter the element in position 1,0: 3
Enter the element in position 1,1: 4
The sum of the elements is 10
```

Complejidad O(LOG N).
Ejemplos de cálculo de complejidad de tiempo

- EN EL SIGUIENTE C DIGO, HAY UN BUCLE WHILE ADEM S DE LA INICIALIZACI N, LAS INSTRUCCIONES DE ENTRADA Y SALIDA, UNA OPERACI N ARITM TICA Y UNA INSTRUCCI N RETURN.

- EXAMINEMOS MATEM TICAMENTE C MO FUNCIONA ESTO MIENTRAS QUE EL BUCLE AQU .
  - SUPONGAMOS N=8; ENTONCES ESTAMOS OBTENIENDO SERIES DE 8 4 2 1
  - DEL MISMO MODO, SI INTENTAMOS DIFERENTES ENTRADAS COMO 10, OBTENDREMOS 10 5 2 1
  - PODEMOS VER QUE EN CADA ITERACI N, EL N MERO SE REDUCIR A LA MITAD ANTERIOR HASTA QUE SE VIOLE TODA LA CONDICI N.
- TALES ALGORITMOS DE DIVIDIR UN N MERO O CONJUNTO DE N MEROS EN MITADES CAEN BAJO LA COMPLEJIDAD LOGAR TMICA O(LOG N).

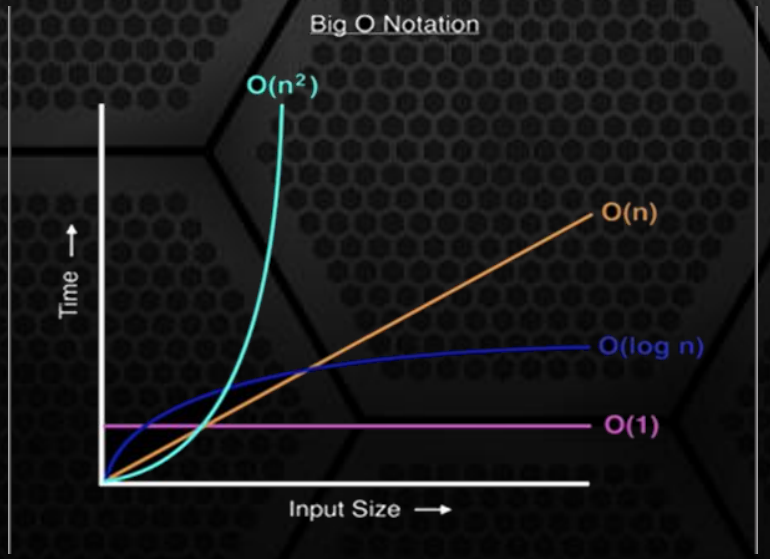
```
#include <iostream>
using namespace std;

int main()
{
int N;
cout<<"Enter a number: ";
cin>>N;
while(n>0)
{
    cout<<N<<" ";
    N=N/2;
}
return 0;
}
```

Salida:

Enter a number: 8
8 4 2 1

NOTA: ADEM S, PUEDE PASAR POR EL ALGORITMO DE B SQUEDA BINARIA.QUE TARDA O(LOGN) EN COMPRENDER LA COMPLEJIDAD LOGAR TMICA EN PROFUNDIDAD. LA LISTA NO TERMINA AQU ; TENEMOS TIEMPO LOG-LINEAL O(NLOGN), COMPLEJIDAD DE TIEMPO C BICO O(N^3), COMPLEJIDAD TEMPORAL EXPONENCIAL O(2^N), COMPLEJIDAD DE TIEMPO FACTORIAL O(N!) Y MUCHOS M S.



2. ¿Qué es la complejidad espacial?

¿Qué es la complejidad espacial?

La complejidad espacial se refiere a la cantidad de memoria requerida en un sistema para un algoritmo en particular. Ahora, antes de profundizar en el concepto, debemos saber qué es el espacio auxiliar.

El espacio auxiliar es el espacio extra utilizado por un algoritmo, que al sumar con el espacio ocupado por los valores de entrada, nos da complejidad espacial.

Complejidad espacial = Espacio auxiliar + Espacio ocupado por los valores de entrada

Nota: La complejidad del espacio no es la suma directa del espacio auxiliar y el espacio ocupado por los valores de entrada. Es el valor máximo posible de la suma para un algoritmo en particular.

Para comprender mejor la complejidad del espacio, consideremos el ejemplo simple de imprimir la serie de Fibonacci con y sin recursión.

Imprimir la serie de Fibonacci con y sin recursión

SIN RECURSIÓN

```
#include<iostream>
using namespace std;

int main()
{
     int a = 0, b = 1, c,i,num;
     cout<<"Enter the number of elements: ";
     cin>>num;
     cout<<a<<" "<<b<<" ";
     for(i=2;i<num;i++)
     {
          c = a + b;
          cout<<c<<" ";
          a=b;
          b=c;
     }
     return 0;
}
```

Salida:
Enter the number of terms of series: 5
0 1 1 2 3

CON RECURSIÓN

```
#include<iostream>
using namespace std;
int Fibonacci(int n)
{
    if((n==1)||(n==0))
    return(n);
    else
    {
return(Fibonacci(n-1)+Fibonacci(n-2));
    }
}
int main()
{
    int num , i=0;
    cout<<"Enter the number of terms of series : ";
    cin>>num;
    while(i < num)
    {
        cout<<" "<< Fibonacci(i);
   i++;
    }
    return 0;
}
```

Code

AUNQUE LAS SALIDAS SON IDÉNTICAS EN LOS DOS FRAGMENTOS DE CÓDIGO MOSTRADOS, EL CÓDIGO CON RECURSIÓN CONSUME MÁS MEMORIA QUE EL QUE NO TIENE RECURSIÓN.
LA RAZÓN DETRÁS DE UN MAYOR USO DE MEMORIA ES LA PILA (STACK) DE RECURSIÓN.

complejidad espacial

¿Cómo encontrar la complejidad del espacio?

LOS SIGUIENTES PASOS DESCRIBEN C MO ENCONTRAR LA COMPLEJIDAD ESPACIAL DE UN ALGORITMO:

1. ENCUENTRE EL N MERO DE VARIABLES DE DIFERENTES TIPOS DE DATOS INICIALIZADOS EN EL ALGORITMO.
2. ENCUENTRE LA MEMORIA REQUERIDA POR CADA TIPO DE DATOS.
3. MULTIPLIQUE LA MEMORIA REQUERIDA POR CADA TIPO DE DATOS CON EL N MERO DE VARIABLES DE ESE TIPO DE DATOS PRESENTES. EL RESULTADO DAR LA MEMORIA REQUERIDA POR CADA TIPO DE DATOS.
4. AGREGUE LA MEMORIA REQUERIDA POR CADA TIPO DE DATOS.
5. IGNORAR LAS CONSTANTES (COMO SE EXPLICA EN LA COMPLEJIDAD DEL TIEMPO), SI LA HAY, EN LA SUMA PARA OBTENER LA COMPLEJIDAD ESPACIAL EN LA NOTACI N BIG O.

AHORA QUE SABEMOS C MO ENCONTRAR LA COMPLEJIDAD ESPACIAL VEAMOS ALGUNOS EJEMPLOS PARA ENTENDER MEJOR.

Complejidad O(1)
Ejemplos de cálculo de complejidad espacial

- EN EL C DIGO MOSTRADO, SE UTILIZAN TRES VARIABLES DE TIPO ENTERO. AHORA, EL TAMA O DE LAS VARIABLES DE TIPO ENTERO SUELE SER DE 2 O 4 BYTES DEPENDIENDO DEL COMPILADOR.
- SI ASUMIMOS QUE SON 4 BYTES EN NUESTRO CASO, LA MEMORIA TOTAL REQUERIDA PARA EL C DIGO ES DE 4 X 3 = 12 BYTES, QUE ES UNA CONSTANTE. AS QUE LA COMPLEJIDAD ESPACIAL ES O(1).

```
#include<iostream>
using namespace std;

int main()
{
  int a, b, c;
  cout<<"Enter the value of a and b ";
  cin>>a>>b;
  c = a + b;
  cout<<"The sum is "<<c;
}
return 0;
```

Complejidad O(N)
Ejemplos de cálculo de complejidad espacial

- AQU , LA MATRIZ CONTIENE N N MERO DE ELEMENTOS, Y HAY TRES VARIABLES DE TIPO ENTERO M S (N, SUMA E I).
- AS QUE LA MEMORIA TOTAL REQUERIDA AQU ES [(4 X N) + (4 X 3)] = (4N + 12) BYTES.
- EN NOTACI N ASINT TICA, LA COMPLEJIDAD DEL ESPACIO ES O(N).

SER A MEJOR PROBAR EJEMPLOS COMPLEJOS SOBRE LA COMPLEJIDAD ESPACIAL, COMO ENCONTRAR LA COMPLEJIDAD ESPACIAL EN MERGE-SORT (O(N)).

```
#include<iostream>
using namespace std;

int main()
{
int n, sum = 0;
cout<<"Enter the number of elements ";
cin>>n;
int arr[n];
      for(int i = 0; i < n; i++)
   {
cout<<"Enter the element in position "<<i<<" ";
cin>>arr[i];
sum = sum + arr[i];
   }
   cout<<sum;
}
```

Comparativa Tiempo & Espacio

En Búsqueda de Solución correcta

Importancia de la complejidad del tiempo y el espacio en las estructuras de datos

Los desarrolladores de programas del mundo real están limitados por la memoria física de las plataformas en las que planean ejecutarse. Aquí es donde entra en juego la complejidad del espacio, ya que nunca queremos ejecutar una función o proceso que consuma más espacio del que el sistema tiene disponible en un momento dado. Por otro lado, no queremos que nuestras operaciones tomen demasiado tiempo para que se empantanen y ralentifien nuestras aplicaciones.

Echemos un vistazo a la complejidad del tiempo y el espacio en el peor de los casos para las operaciones de estructura de datos comunes a continuación:

ESTRUCTURAS DE DATOS	COMPLEJIDAD DEL TIEMPO	COMPLEJIDAD ESPACIAL			
	ACCESO	BUSCAR	INSERCI N	DELECI N	
ARREGLO DE DISCOS	O(1)	O(N)	O(N)	O(N)	O(N)
PILA	O(N)	O(N)	O(1)	O(1)	O(N)
COLA	O(N)	O(N)	O(1)	O(1)	O(N)
LISTA DE ENLAZADOS POR S SOLOS	O(N)	O(N)	O(1)	O(1)	O(N)
LISTA DOBLEMENTE VINCULADA	O(N)	O(N)	O(1)	O(1)	O(N)
TABLA HASH	O(N)	O(N)	O(N)	O(N)	O(N)
RBOL DE B SQUEDA BINARIA	O(N)	O(N)	O(N)	O(N)	O(N)
RBOL BINARIO	O(N)	O(N)	O(N)	O(N)	O(N)

Resumen - Comparativa Tiempo & Espacio

QU SE ENTIENDE POR LA COMPLEJIDAD DEL TIEMPO DE UN ALGORITMO?

LA COMPLEJIDAD DEL TIEMPO SE REFIERE A LA CANTIDAD DE TIEMPO REQUERIDO POR EL ALGORITMO O PROGRAMA PARA EJECUTARSE HASTA SU FINALIZACI N.

C MO SE CALCULA LA COMPLEJIDAD DEL TIEMPO Y LA COMPLEJIDAD DEL ESPACIO?

PODEMOS CALCULAR LA COMPLEJIDAD DEL TIEMPO Y EL ESPACIO ENCONTRANDO EL TIEMPO DE EJECUCI N Y LA MEMORIA TOTAL TOMADA POR EL ALGORITMO.

QU COMPLEJIDAD ESPACIAL ES LA MEJOR?

O(1) LA COMPLEJIDAD ESPACIAL ES LA MEJOR.

CU L ES LA MEJOR COMPLEJIDAD HORARIA?

LA MEJOR COMPLEJIDAD DE TIEMPO ES O(1). POR EJEMPLO, EN EL CASO DE LA B SQUEDA LINEAL, EL MEJOR CASO SER A SI EL ELEMENTO SE ENCUENTRA EN LA PRIMERA ITERACI N, ES DECIR, LA COMPLEJIDAD TEMPORAL SER A O(1).

Resumen - Comparativa Tiempo & Espacio

QU ES LA NOTACI N BIG O EN UN ALGORITMO?

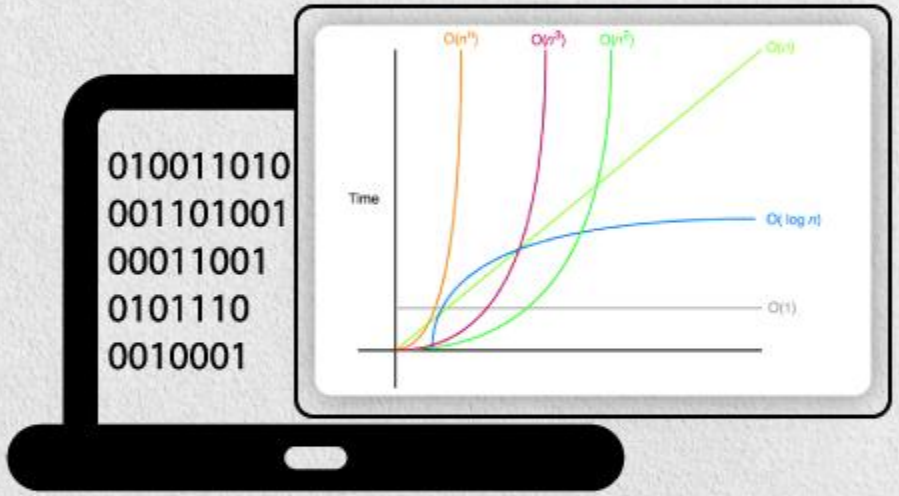
EN LA NOTACI N BIG O, LA COMPLEJIDAD TEMPORAL SE DEFINE UTILIZANDO UN L MITE SUPERIOR ASINT TICO, ES DECIR, CONSIDERANDO EL PEOR DE LOS CASOS DONDE N TIENDE AL INFINITO. UN EJEMPLO QUE MUESTRA LA NOTACI N BIG O SE DA ANTES.

LA COMPLEJIDAD DEL ESPACIO INCLUYE LA ENTRADA?

S , LA COMPLEJIDAD ESPACIAL INCLUYE TANTO EL ESPACIO AUXILIAR COMO EL ESPACIO UTILIZADO POR LA ENTRADA.

HAY ALGUNA DIFERENCIA ENTRE LA COMPLEJIDAD DEL TIEMPO Y EL TIEMPO DE EJECUCI N?

COMPLEJIDAD DEL TIEMPO	TIEMPO DE EJECUCI N
ES UN CONCEPTO TE RICO QUE DA UN COMPORTAMIENTO ASINT TICO	ES EL TIEMPO QUE UN PROGRAMA TARDA EN EJECUTARSE
DEPENDE SOLO DEL ALGORITMO.	DEPENDE DEL LENGUAJE DE PROGRAMACI N, LAS BIBLIOTECAS UTILIZADAS Y PAR METROS COMO LA VELOCIDAD DEL PROCESADOR Y LA MEMORIA.



Understanding Big O notation

BIG-O COMPLEXITIES

OF COMMON ALGORITHMS USED IN COMPUTER SCIENCE

DATA STRUCTURE OPERATIONS

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	O(1)	O(n)	O(n)	O(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Stack	O(n)	O(n)	O(1)	O(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Singly-Linked List	O(n)	O(n)	O(1)	O(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly-Linked List	O(n)	O(n)	O(1)	O(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Skip List	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
Hash Table	-	O(1)	O(1)	O(1)	-	O(n)	O(n)	O(n)	O(n)
Binary Search Tree	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	-	O(log(n))	O(log(n))	O(log(n))	-	O(n)	O(n)	O(n)	O(n)
B-Tree	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Red-Black Tree	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Splay Tree	-	O(log(n))	O(log(n))	O(log(n))	-	O(log(n))	O(log(n))	O(log(n))	O(n)
AVL Tree	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)

ARRAY SORTING ALGORITHMS

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	O(n log(n))	O(n log(n))	O(n^2)	O(log(n))
Mergesort	O(n log(n))	O(n log(n))	O(n log(n))	O(n)
Timsort	O(n)	O(n log(n))	O(n log(n))	O(n)
Heapsort	O(n log(n))	O(n log(n))	O(n log(n))	O(1)
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)
Shell Sort	O(n)	O((nlog(n))^2)	O((nlog(n))^2)	O(1)
Bucket Sort	O(n+k)	O(n+k)	O(n^2)	O(n)
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)

LEGEND

- Excellent
- Good
- Fair
- Bad
- Horrible

GRAPH OPERATIONS

Node / Edge Management	Storage	Add Vertex	Add Edge	Remove Vertex	Remove Edge	Query
Adjacency list	O(|V|+|E|)	O(1)	O(1)	O(|V|+|E|)	O(|E|)	O(|V|)
Incidence list	O(|V|+|E|)	O(1)	O(1)	O(|E|)	O(|E|)	O(|E|)
Adjacency matrix	O(|V|^2)	O(|V|^2)	O(1)	O(|V|^2)	O(1)	O(1)
Incidence matrix	O(|V| · |E|)	O(|V| · |E|)	O(|V| · |E|)	O(|V| · |E|)	O(|V| · |E|)	O(|E|)

HEAP OPERATIONS

Heap Type	Time Complexity						
	Heapify	Find Max	Extract Max	Increase Key	Insert	Delete	Merge
Linked List (sorted)	–	O(1)	O(1)	O(n)	O(n)	O(1)	O(m+n)
Linked List (unsorted)	–	O(n)	O(n)	O(1)	O(1)	O(1)	O(1)
Binary Heap	O(n)	O(1)	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(m+n)
Binomial Heap	–	O(1)	O(log(n))	O(log(n))	O(1)	O(log(n))	O(log(n))
Fibonacci Heap	–	O(1)	O(log(n))	O(1)	O(1)	O(log(n))	O(1)

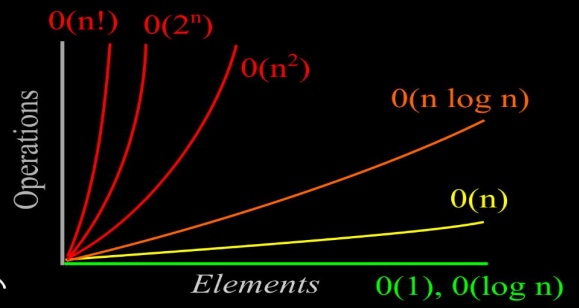
<BIG-O-CHEATSHEET>

Legend

TIME Complexity vs. SPACE Complexity

Good Fair Bad

Good Fair Bad



DATA STRUCTURE Operations

DATA Structure	TIME Complexity								SPACE Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	O(1)	O(n)	O(n)	O(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Stack	O(n)	O(n)	O(1)	O(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	O(n)	O(n)	O(1)	O(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Singly-Linked List	O(n)	O(n)	O(1)	O(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly-Linked List	O(n)	O(n)	O(1)	O(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Skip List	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
Hash Table	N/A	O(1)	O(1)	O(1)	N/A	O(n)	O(n)	O(n)	O(n)
Binary Search Tree	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	N/A	O(log(n))	O(log(n))	O(log(n))	N/A	O(n)	O(n)	O(n)	O(n)
B-Tree	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Red-Black Tree	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Splay Tree	N/A	O(log(n))	O(log(n))	O(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))	O(n)
AVL Tree	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
KD Tree	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)

ARRAY SORTING Algorithms

ARRAY Algorithms	TIME Complexity			SPACE Complexity
	Best	Average	Worst	Worst
Quicksort	O(n log(n))	O(n log(n))	O(n^2)	O(log(n))
Mergesort	O(n log(n))	O(n log(n))	O(n log(n))	O(n)
Timsort	O(n)	O(n log(n))	O(n log(n))	O(1)
Heapsort	O(n log(n))	O(n log(n))	O(n log(n))	O(1)
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)
Tree Sort	O(n log(n))	O(n log(n))	O(n^2)	O(n)
Shell Sort	O(n log(n))	O(n log(n)^2)	O(n log(n)^2)	O(1)
Bucket Sort	O(n+k)	O(n+k)	O(n^2)	O(n)
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)
Cubesort	O(n)	O(n log(n))	O(n log(n))	O(n)

Resumen

La notación **Big O** es una de las notaciones matemáticas más necesarias utilizadas en informática para medir la eficiencia de un algoritmo. Podemos analizar qué tan eficiente es un algoritmo a partir de la cantidad de tiempo, almacenamiento, otros recursos que se necesitan para ejecutar el algoritmo y un cambio en el tamaño de entrada. La notación Big O en la estructura de datos nos dice qué tan bien funcionará un algoritmo en una situación particular. En otras palabras, proporciona el tiempo de ejecución de límite superior de un algoritmo o la complejidad del peor de los casos.

Las notaciones asintóticas

El rendimiento de un algoritmo puede cambiar con un cambio en el tamaño de entrada. Ahí es donde entran en juego las notaciones asintóticas como Big O Notation. Las notaciones asintóticas pueden describir el tiempo de ejecución de un algoritmo cuando la entrada tiende hacia un valor específico o limitante. El análisis asintótico ayuda a analizar el cambio de rendimiento del algoritmo en el orden del tamaño de entrada.

¿Qué es la notación Big O en la estructura de datos?

La notación Big O en la estructura de datos se utiliza para expresar la complejidad algorítmica utilizando términos algebraicos. Describe el límite superior del tiempo de ejecución de un algoritmo y calcula el tiempo y la cantidad de memoria necesarios para ejecutar el algoritmo para un valor de entrada.

Resumen

Propiedades de la notación Big O
Las propiedades más importantes de Big O Notation en la estructura de datos son:

Multiplicación constante:
Si f(n) = CG(n), entonces O(f(n)) = O(g(n)) para una constante c > 0

Función de suma:
Si f(n) = f1(n) + f2(n) + -- + FM(n) y fi(n)≤ fi+1(n) ∀ i=1, 2, --, m,

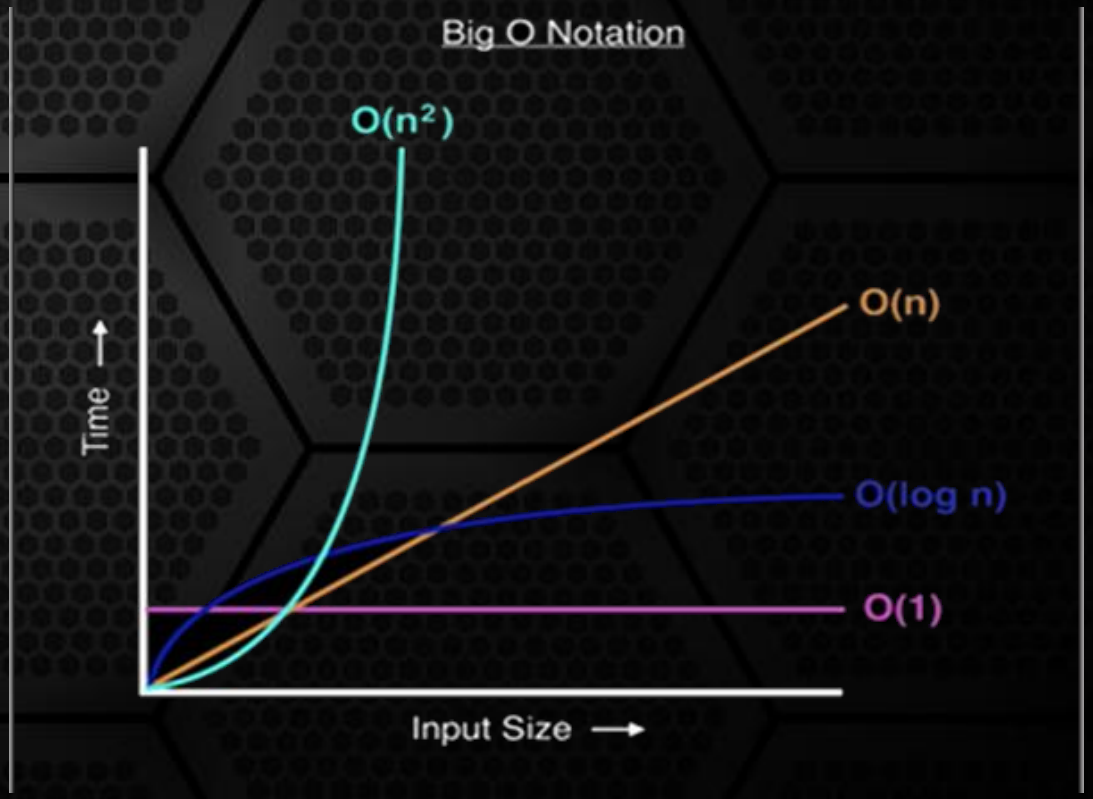
entonces O(f(n)) = O(max(f1(n), f2(n), --, fm(n)))

Función logarítmica:
Si f(n) = log an y g(n)=log bn, entonces

O(f(n)) = O(g(n))

Función polinómica:
Si f(n) = a0 + a1.n + a2.n2 + -- + am.nm, entonces

O(f(n)) = O(nm)



Resumen

¿Cómo hace Big O Notation un análisis en tiempo de ejecución de un algoritmo?
Para analizar y calcular el rendimiento de un algoritmo, debemos calcular y comparar las complejidades de tiempo de ejecución del algoritmo en el peor de los casos. El orden de O(1), conocido como tiempo de ejecución constante, es el tiempo de ejecución más rápido para un algoritmo, y el tiempo que tarda el algoritmo es igual para diferentes tamaños de entrada. Aunque el tiempo de ejecución constante es el tiempo de ejecución ideal para un algoritmo, rara vez se puede lograr porque el tiempo de ejecución depende del tamaño de n ingresado.

¿Cómo analiza la notación Big O la complejidad del espacio?
También es esencial determinar la complejidad espacial de un algoritmo. Esto se debe a que la complejidad del espacio indica cuánto espacio de memoria ocupa el algoritmo. Comparamos las complejidades espaciales del algoritmo en el peor de los casos.

Antes de que la notación Big O analice la complejidad del espacio, se deben implementar las siguientes tareas:

Implementación del programa para un algoritmo en particular.
El tamaño de la entrada n debe conocerse para calcular la memoria que contendrá cada elemento.
Complejidades espaciales de algunos algoritmos comunes:

Búsqueda lineal, búsqueda binaria, ordenación por burbujas, ordenación por selección, ordenación por montones, ordenación por inserción: la complejidad del espacio es O(1).

Ordenación de radix: la complejidad del espacio es O(n+k).
Ordenación rápida: la complejidad del espacio es O(n).
Ordenar combinación: la complejidad del espacio es O(log n).

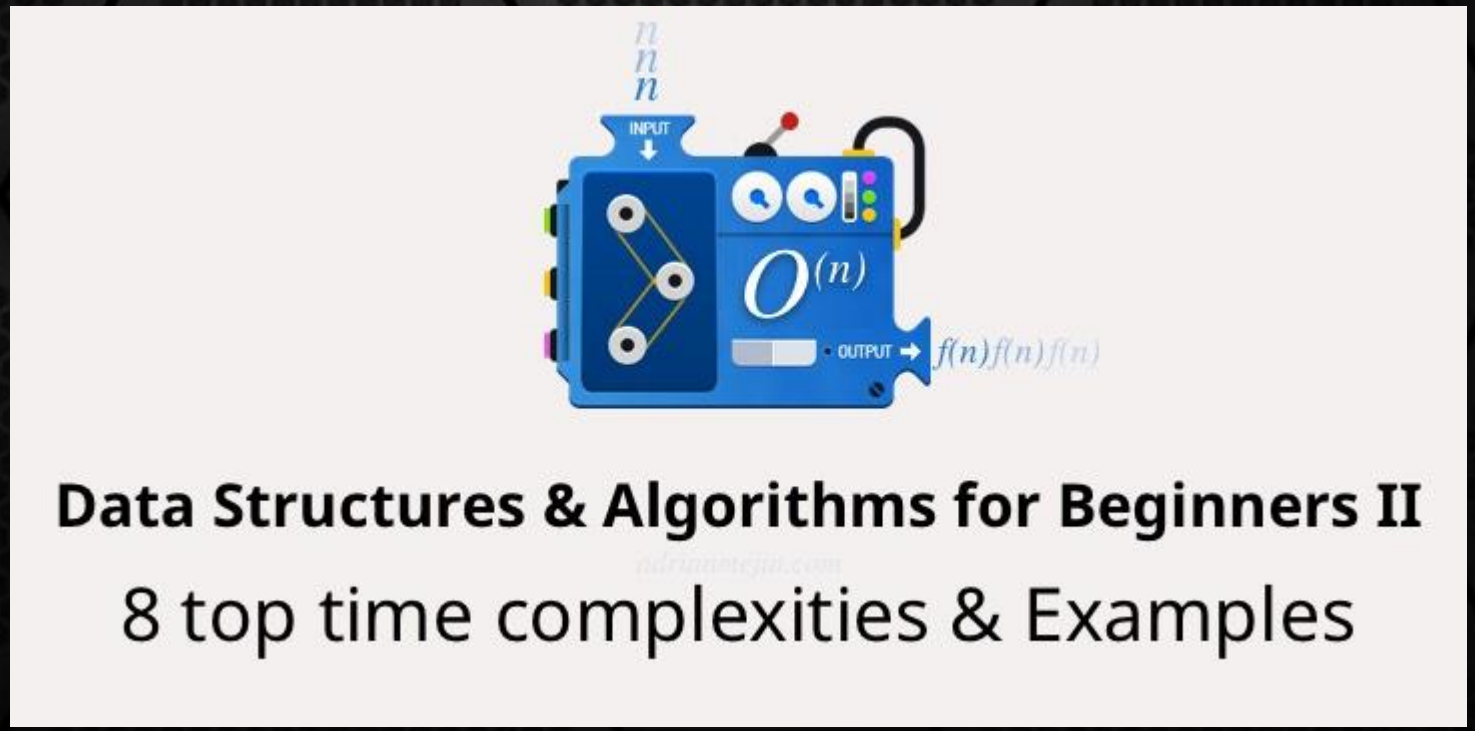
Conclusión:

Estas son las principales conclusiones:

•La velocidad del algoritmo no se mide en segundos, pero la velocidad se mide en el crecimiento del número de operaciones.
•En su lugar, hablamos de la rapidez con la que el tiempo de ejecución de un algoritmo aumenta / disminuye a medida que aumenta / disminuye el tamaño de la entrada.
•La notación Big O es útil para expresar el tiempo de ejecución de los algoritmos.
•O(log n) es más rápido que O(n), pero se vuelve mucho más rápido ya que los elementos de la lista son enormes / aumentan drásticamente.

Muchas Gracias.

Material Opcional Extra



hoja de trucos de O

Notación Big O	Nombre	Ejemplo(s)
O(1)	Constante	# Número impar o par, # Tabla de búsqueda (en promedio)
O(log n)	Logarítmico	# Elemento de búsqueda en matriz ordenada con **búsqueda binaria**
O(n)	Lineal	# Buscar elemento máximo en matriz sin ordenar, # Duplicar elementos en matriz con Hash Map
O(n log n)	Linealitmico	# Ordenar elementos en matriz con **ordenar de combinación**
$O(n^2)$	Cuadrático	# Duplicar elementos en array **(naïve)**, # Sorting array with bubble **sort**
$O(n^3)$	Cúbico	# solucionador de ecuaciones de 3 variables
$O(2^n)$	Exponencial	# Buscar todos los subconjuntos
O(n!)	Factorial	# Buscar todas las permutaciones de un conjunto/cadena determinado